

软件工程实践

《系统设计说明书》

项目名称： 知音短视频平台

开发团队： 因为把 dll 看成 ddl 而急死

学号姓名： 102300229 叶达

学号姓名： 102300225 吴嘉鑫

学号姓名： 102300221 苏峻

学号姓名： 102300202 陈雨桐

学号姓名： 102300109 郑天浩

学号姓名： 032201218 陈彦哲

学号姓名： 102300206 张钰婷

学号姓名： 102300127 李严

学号姓名： 032201109 朱添驰

学号姓名： 102300214 陈雨昕

2025 年 10 月 19 日

目录

1 文档介绍 5
1.1 文档目的 5
1.2 阅读对象 5
1.3 术语与缩写解释 6
1.4 文档的约定 8
1.4.1 格式约定 8
1.4.2 接口描述约定 8
1.5 参考资料 9
2.总体设计 9
2.1 系统运行环境 9
2.1.1 硬件设备配置清单 10
2.1.2 软件系统配置清单 11
2.2 系统开发环境 13
2.2.1 开发工具配置清单 13
2.2.2 开发环境配置要求 14
2.3. 软件结构 14
2.3.1 系统架构图 14
2.3.2 架构层次说明 15
2.3.3 数据流向说明 16
3 系统功能模块 16
3.1 用户模块 16
3.1.1 功能模块描述及流程 16
3.1.2 模块设计说明 20
3.2 视频模块 21
3.2.1 功能模块描述及流程 21
3.2.2 模块设计说明 25

3.3 社交互动模块 26
3.3.1 功能模块描述及流程 26
3.3.2 模块设计说明 32
3.4 笔记模块 32
3.4.1 功能模块描述及流程 32
3.4.2 模块设计说明 36
3.5 内容消费与推荐模块 37
3.5.1 功能模块描述及流程 37
3.5.2 模块设计说明 44
3.6 核心业务流程时序图 46
3.6.1 用户注册登录时序图 46
3.6.2 视频上传发布时序图 47
3.6.3 社交互动时序图 48
3.6.4 个性化推荐时序图 49
4 用例图 50
5 数据流图 51
5.1 数据流图顶层图和底层图 51
6 接口设计 52
6.1 接口总览 52
6.1.1 接口命名规范 52
6.1.2 响应状态码规定 52
6.1.3 注意事项 53
6.1.4 协议约定 54
6.2 后端接口文档 55
6.2.1 用户模块 55
6.2.2 视频模块 59
7 权限设计 63

7.1 基于角色的 API 访问控制 63
7.2 数据库权限管理 65
7.3 文件系统权限控制 66
7.4 缓存和消息队列权限 66
7.5 安全增强机制 67
8 安全性、健壮性与可扩展性 67
8.1 安全性 67
8.2 健壮性 68
8.3 可扩展性 69

1 文档介绍

1.1 文档目的

《知音》App 系统设计说明书旨在为项目全生命周期相关人员提供统一、详细的系统设计依据，具体目标包括：

1. 明确系统整体架构（含微服务拆分、技术栈选型、数据流向），指导后端、前端、移动端开发团队开展技术实现；
2. 规范功能模块设计（用户、视频、社交互动、笔记、内容推荐等），明确各模块的业务逻辑、输入输出要求及异常处理机制；
3. 定义接口标准（RESTful 风格、权限控制、响应格式）、数据存储规则（数据库表结构、缓存策略）及安全设计（加密、防攻击），确保各模块协同兼容；
4. 为测试团队提供测试范围与验收依据，为运维团队提供部署配置（单机 / 生产环境）、监控告警设计参考；
5. 作为项目评审、版本迭代（如后续新增直播、付费专栏功能）的基础文档，保障业务需求与技术实现的一致性。

1.2 阅读对象

阅读对象群体	核心关注内容
后端开发团队	系统架构图、微服务拆分（如用户服务、推荐服务）、接口设计（URL、请求 / 响应格式）、数据库权限管理、中间件（Redis/Kafka）配置
前端 / 移动端开发团队	客户端层设计（Flutter 跨平台实现）、功能模块流程（如视频上传 / 播放活动图）、接口调用规范、前端权限控制逻辑
测试团队	功能模块输入输出要求、异常处理机制（错误码）、核心业务流程时序图（如用户注册登录）、健壮性设计（容错 / 降级策略）
运维团队	系统运行环境（硬件 / 软件配置清单）、部署方案（前期单机 / 生产分布式）、监控系统

	（Prometheus+Grafana）、可扩展性设计（分库分表）
产品经理	功能模块描述（如笔记类型定义、内容推荐逻辑）、用户交互流程（如社交互动活动图）、业务规则（如用户角色权限）
项目管理人员	文档更改记录、模块开发优先级（如核心视频模块 vs 辅助笔记模块）、技术风险点（如数据泄露应对）
技术评审专家	架构合理性（分层设计、微服务解耦）、技术栈选型适配性（Go+Hertz vs 其他框架）、安全与健壮性设计完整性

1.3 术语与缩写解释

术语 / 缩写	英文全称	解释说明
微服务架构	Microservice Architecture	将系统拆分为独立的服务模块（如用户服务、视频服务），各模块可独立部署、扩展，通过 RPC / 消息队列通信，降低耦合度
JWT	JSON Web Token	用于用户身份认证的轻量级令牌，包含用户 ID、角色等信息，有效期 2 小时，通过请求头 Authorization: Bearer {Token}传递
RBAC	Role-Based Access Control	基于角色的访问控制模型，通过 “角色 - 权限” 映射（如 “普通用户 - 视频上传权限”“管理员 - 内容审核权限”）实现精细化权限管理
RESTful	Representational State Transfer	接口设计风格，通过 HTTP 方法（GET 查询、POST 创建、PUT 全量更新、PATCH 部分更新、DELETE 删除）对应资源操作，URL 采用小写 + 下划线命名

CDN	Content Delivery Network	内容分发网络，将视频、图片等静态资源部署至全国节点，降低用户访问延迟（如《知音》App 中视频流通过 CDN 分发）
MySQL	-	关系型数据库，用于存储结构化数据（如用户信息、视频元数据），采用主从复制架构保障高可用性
Redis	Remote Dictionary Server	内存数据库，用于缓存热点数据（如用户会话、热门视频列表）、实现分布式锁，提升系统响应速度
Kafka	-	高吞吐量消息队列，用于处理异步任务（如用户行为日志收集、视频转码任务分发），支持解耦服务
Elasticsearch	-	搜索引擎，用于实现全文搜索（如笔记内容搜索、视频关键词检索）及个性化推荐数据处理
Flutter	-	跨平台开发框架，用于开发《知音》App 的 iOS/Android 客户端，实现 “一次开发、多端运行”
Hertz	-	字节跳动开源的 Go 语言微服务框架，用于《知音》App 后端服务开发，支持高并发、低延迟
QPS	Queries Per Second	每秒查询量，衡量系统接口处理能力，《知音》App 要求核心接口（如视频播放）QPS 支持 1000+
WAF	Web Application Firewall	Web 应用防火墙，用于拦截 SQL 注入、XSS 等网络攻击，保障《知音》App API 接口安全
TLS 1.3	Transport Layer Security 1.3	传输层加密协议，用于敏感数据（如手机号、密码）传输加密，防止数据泄露

1.4 文档的约定

1.4.1 格式约定

1. **文本格式：**1 级标题二号等线字体，加粗；2 级标题三号等线字体，加粗；3 级标题小四等线字体，加粗；正文 11 号等闲字体。

1.4.2 接口描述约定

2. **URL 命名**：遵循 RESTful 风格，格式为/api/v{版本号}/{模块名}/{资源名}/{操作}（操作可选），示例：
    1. 用户注册：/api/v1/user/register（POST）
    2. 获取视频详情：/api/v1/video/get_detail（GET）
    3. 更新笔记：/api/v1/note/update（PATCH）
3. **HTTP 方法映射**：
    1. GET：查询资源（如获取用户详情、视频列表）
    2. POST：创建资源（如注册、上传视频）
    3. PUT：全量更新资源（如替换用户资料）
    4. PATCH：部分更新资源（如修改视频标题）
    5. DELETE：删除资源（如删除笔记、取消关注）
4. **响应格式**：所有接口返回 JSON 格式数据，统一包含 code（状态码）、message（提示信息）、data（业务数据）、timestamp（时间戳）四字段，示例：

```
{
"code": 200,
"message": "操作成功",
"data": {
"user_id": "u123456",
"nickname": "知音用户"
},
"timestamp": 1738704366100
```

```
}
```

5. **错误码规则**：错误码为 5 位数字，前 2 位标识错误类型，后 3 位为具体场景，示例：
    1. 400xx：客户端参数错误（如 40001 “手机号已注册”）
    2. 401xx：认证授权错误（如 40101 “令牌过期”）
    3. 500xx：服务端业务错误（如 50001 “不支持的视频格式”）
    4. 800xx：推荐 / 搜索服务错误（如 80001 “推荐服务异常”）

1.5 参考资料

资料名称	来源
《系统设计说明书》国际规范文本	-
需求规格说明书	因为把 dll 看成 ddl 而急死队
团队选题报告	因为把 dll 看成 ddl 而急死队
数据库设计说明书	因为把 dll 看成 ddl 而急死队

2.总体设计

2.1 系统运行环境

《知音》App 采用现代化的微服务架构设计，将复杂的业务系统拆分为多个独立的服务模块，每个模块专注于特定的业务功能领域。这种架构设计不仅提高了系统的可维护性和可扩展性，还便于团队分工协作和独立部署。

整个系统采用前后端分离的设计模式，构建了一个完整的技术栈体系。后端基于 Go 语言开发，选用 Hertz 框架构建高性能的微服务。前端采用 Flutter 框架实现跨平台移动应用开发，在数据存储方面，系统结合使用 MySQL 关系型数据库和 Redis 缓存数据库，既保证了数据的一致性和完整性，又提升了访问性能。服务间的异步通信通过 Kafka 消息队列实现，确保了系统的高可用性和解耦性。同时，系统集成 Elasticsearch 实现

强大的全文搜索和个性化推荐功能，为用户提供更好的内容发现体验。

2.1.1 硬件设备配置清单

考虑到项目初期的资源限制和部署成本，我们提供两种硬件配置方案：前期单机部署方案和生产环境部署方案。

2.1.1.1 前期单机部署方案

适用于项目初期开发、测试和小规模用户验证阶段，所有服务部署在一台服务器上。

设备类型	配置要求	数量	用途说明
开发测试服务器	CPU: 4 核, 内存: 16GB, 硬盘: 2TB SSD	1 台	集成运行所有微服务、数据库、缓存、消息队列等
文件存储服务器	CPU: 2 核, 内存: 8GB, 硬盘: 5TB	1 台	存储视频、图片等大文件（可选，初期可与主服务器共用存储）

2.1.1.2 生产环境部署方案

适用于正式上线后的生产环境，采用分布式部署以保证系统的高可用性和可扩展性。

设备类型	配置要求	数量	用途说明
应用服务器	CPU: 8 核以上, 内存: 16GB 以上, 硬盘: 500GB SSD	3 台	运行微服务应用
数据库服务器	CPU: 16 核以上, 内存: 32GB 以上, 硬盘: 1TB SSD	2 台	主从 MySQL 数据库
缓存服务器	CPU: 4 核以上, 内存: 16GB 以上, 硬盘: 200GB SSD	2 台	Redis 集群
搜索服务	CPU: 8 核以上, 内存: 32GB 以上,	3	Elasticsearch 集群

器	硬盘: 1TB SSD	台	
消息队列服务器	CPU: 4 核以上, 内存: 8GB 以上, 硬盘: 500GB SSD	3 台	Kafka 集群
文件存储服务器	CPU: 8 核以上, 内存: 16GB 以上, 硬盘: 10TB	3 台	分布式文件存储
负载均衡器	CPU: 4 核以上, 内存: 8GB 以上, 硬盘: 200GB SSD	2 台	请求分发和负载均衡
监控服务器	CPU: 4 核以上, 内存: 16GB 以上, 硬盘: 1TB SSD	1 台	Prometheus + Grafana 监控
网关服务器	CPU: 4 核以上, 内存: 8GB 以上, 硬盘: 200GB SSD	2 台	API 网关服务

2.1.2 软件系统配置清单

本系统采用全开源技术栈构建，充分利用成熟的开源工具来实现各种功能模块。所有软件均选用社区活跃、文档完善、有长期支持的开源项目。

软件类型	软件名称	版本要求	用途说明
操作系统	Ubuntu/CentOS	Ubuntu 20.04 LTS/CentOS 8	服务器操作系统
容器运行时	Docker	20.10 以上	容器化运行环境
容器编排	Kubernetes	1.20 以上	容器编排管理
编程语言	Go	1.23 以上	后端服务开发
数据库	MySQL	8.0 以上	关系型数据存

			储
缓存数据库	Redis	6.0 以上	缓存和会话存储
搜索引擎	Elasticsearch	8.0 以上	全文搜索和推荐
消息队列	Kafka	3.0 以上	异步消息处理
负载均衡	Nginx	1.20 以上	请求负载均衡
API 网关	Hertz Gateway	最新版本	API 网关服务
监控系统	Prometheus	2.30 以上	系统监控
数据可视化	Grafana	8.0 以上	监控数据展示
日志收集	Fluentd	1.14 以上	日志收集和处理
推荐引擎	Apache Spark MLlib	3.0 以上	机器学习推荐算法
推荐引擎	TensorFlow Serving	2.0 以上	深度学习推荐模型服务
推荐引擎	Faiss	1.7 以上	向量相似度搜索
搜索引擎	Elasticsearch IK 分词器	8.0 以上	中文分词处理
搜索引擎	Elasticsearch analysis-	最新版本	阿里云中文分

	aliyun		析插件
对象存储	MinIO	最新版本	分布式文件存储
服务网格	Istio	1.10 以上	微服务间通信管理
链路追踪	Jaeger	1.20 以上	分布式系统调用链追踪
服务注册发现	Consul	1.10 以上	服务注册与发现

2.2 系统开发环境

2.2.1 开发工具配置清单

为了确保开发效率和代码质量，团队需要配置一套完整的开发工具链。开发环境以 GoLand 或 VS Code 作为主要 IDE，Git 作为代码版本控制工具，Go Modules 进行依赖包管理。构建和测试方面使用 Make 工具和 Go Test，文档生成采用 Swagger。在容器化开发方面，使用 Docker Desktop 进行本地容器测试，Minikube 或 k3d 进行本地 Kubernetes 环境搭建。数据库设计和管理使用 MySQL Workbench，缓存和消息队列则分别使用 Redis CLI 和 Kafka CLI 进行操作。

工具类型	工具名称	版本要求	用途说明
IDE	GoLand/VS Code	最新版本	Go 语言开发环境
代码管理	Git	2.30 以上	代码版本控制
包管理	Go Modules	Go 1.18 内置	依赖包管理
构建工具	Make	4.0 以上	项目构建和编译

测试工具	Go Test	Go 内置	单元测试和集成测试
文档工具	Swagger	2.0 以上	API 文档生成
容器工具	Docker Desktop	最新版本	本地容器化测试
编排工具	Minikube/k3d	最新版本	本地 Kubernetes 测试
数据库工具	MySQL Workbench	8.0 以上	数据库设计和管理
缓存工具	Redis CLI	6.0 以上	Redis 命令行工具
消息队列工具	Kafka CLI	3.0 以上	Kafka 命令行工具

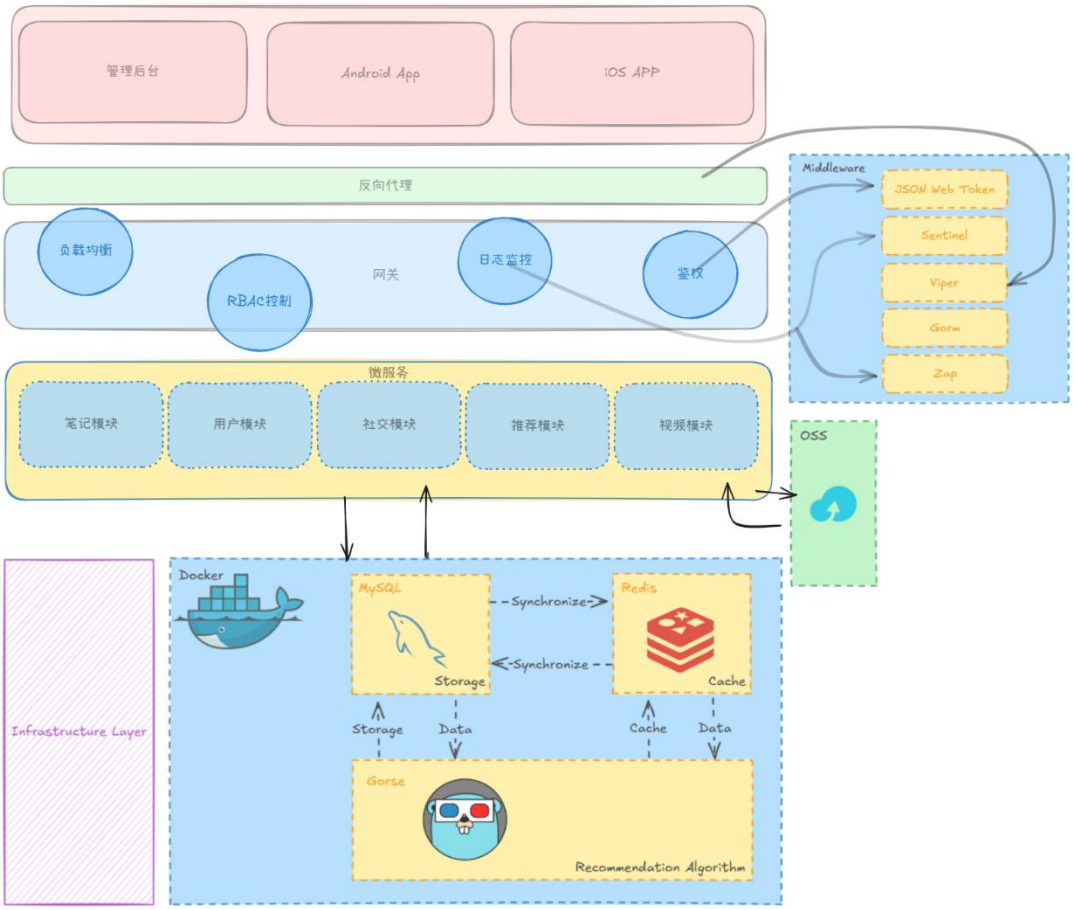
2.2.2 开发环境配置要求

开发流程遵循现代软件工程实践，代码提交采用 Git Flow 工作流管理分支，所有代码在合并前必须通过 Code Review。测试环节集成自动化测试，并通过 GitHub Actions 实现持续集成，确保代码质量和快速交付。

2.3. 软件结构

2.3.1 系统架构图

为了更好地展示《知音》App 的系统架构，我们采用分层架构设计，清晰地展现各组件之间的关系和数据流向。



2.3.2 架构层次说明

架构说明:

系统采用分层架构设计，各组件协同工作，共同支撑《知音》App 的完整功能。

客户端层为用户提供多样化的访问入口，包括基于 Flutter 框架开发的 iOS App 和 Android App，以及功能完善的 Web 管理后台，满足不同用户群体的使用需求。

网关层作为系统的统一入口，承担着关键的流量管控职责。负载均衡器确保请求在多个服务实例间合理分配，提高系统可用性；API 网关则负责请求路由、认证授权、限流熔断等核心功能，为后端服务提供安全可靠的访问控制。

服务层是系统的核心业务逻辑层，包含五个独立的微服务模块。用户服务负责处理账户相关的注册、登录和个人资料管理功能；视频服务专注于视频、文档等媒体内容的上传、存储和管理；推荐服务基于用户行为数据实现个性化内容推荐；社交服务支持关注、点赞、评论等用户互动功能；笔记服务为用户提供笔记的编辑上传、存储、管理

中间件层为系统提供各种基础服务支持，确保各服务间的高效协同。Redis 作为高速缓

存存储热点数据和用户会话信息，提升系统响应速度；Kafka 处理用户行为日志和系统通知等异步消息，保证数据的可靠传输；Elasticsearch 提供全文搜索和推荐数据处理能力，增强用户体验；RabbitMQ 处理其他类型的异步任务，进一步解耦系统组件。

数据存储层负责系统的数据持久化，采用多种存储方案满足不同数据类型的需求。MySQL 数据库存储用户信息、内容元数据等结构化数据，通过主从复制机制保证高可用性；分布式文件存储系统专门用于存储视频、图片等大文件资源，确保媒体内容的高效存取。

基础设施层提供系统运行的基础环境，保障系统的稳定性和可维护性。Docker 实现应用的容器化部署，提高部署效率和环境一致性；docker compose 负责容器编排管理，确保服务的高可用性；Prometheus 持续收集系统监控数据，Grafana 提供直观的监控数据可视化展示，为系统运维提供全面的数据支持。

2.3.3 数据流向说明

在用户请求流程中，用户通过移动应用或 Web 后台发起请求，请求首先到达 API 网关进行身份认证和权限验证。网关根据路由规则将请求转发到对应的服务实例，服务实例处理业务逻辑后，可能需要通过 RPC 调用或消息队列与其他服务进行交互。处理结果数据最终存储到相应的数据库或文件系统中，响应则通过网关返回给客户端。

在数据处理流程中，用户的行为数据通过 Kafka 进行收集，推荐服务实时消费这些数据进行个性化计算。搜索服务将内容数据同步到 Elasticsearch 以支持检索功能，同时 Prometheus 持续收集.系统的监控数据并在 Grafana 中进行可视化展示，为系统运维提供数据支持。

3 系统功能模块

3.1 用户模块

3.1.1 功能模块描述及流程

3.1.1.1 功能概述

用户模块负责管理用户的注册、登录、个人资料管理、权限控制和通知中心等核心功能，是整个系统的基础模块。

3.1.1.2 功能输入输出要求

FR-U-01: 注册与登录

- **输入要求**:

- 手机号：String 类型，长度 11 位，符合中国大陆手机号格式
- 验证码：String 类型，6 位数字
- 密码：String 类型，8-20 位，包含字母和数字

- **输出要求**：
  - 注册成功：返回用户 ID（String 类型）、访问令牌（JWT Token）
  - 登录成功：返回用户基本信息、访问令牌、刷新令牌

FR-U-02: 个人资料管理

- **输入要求**：
  - 头像：图片文件，支持 JPG/PNG 格式，大小 < 5MB
  - 昵称：String 类型，长度 2-20 字符，不含特殊字符
  - 简介：String 类型，长度 < 200 字符
  - 兴趣标签：Array 类型，最多选择 10 个标签
- **输出要求**：
  - 更新成功确认、个人主页数据同步状态

FR-U-03: 通知中心

- **输入要求**：
  - 通知类型过滤：Enum 类型（点赞、评论、关注、系统通知）
  - 分页参数：pageSize（Integer，默认 20），pageNum（Integer，从 1 开始）
- **输出要求**：
  - 通知列表：包含通知 ID、类型、内容、时间戳、已读状态
  - 未读消息数量：Integer 类型

3.1.1.3 业务处理逻辑

用户注册流程：

1. 验证手机号格式和唯一性
2. 发送短信验证码（6 位随机数字，有效期 5 分钟）
3. 验证码校验通过后创建用户记录

4. 生成 JWT 访问令牌和刷新令牌
5. 初始化用户默认设置和权限

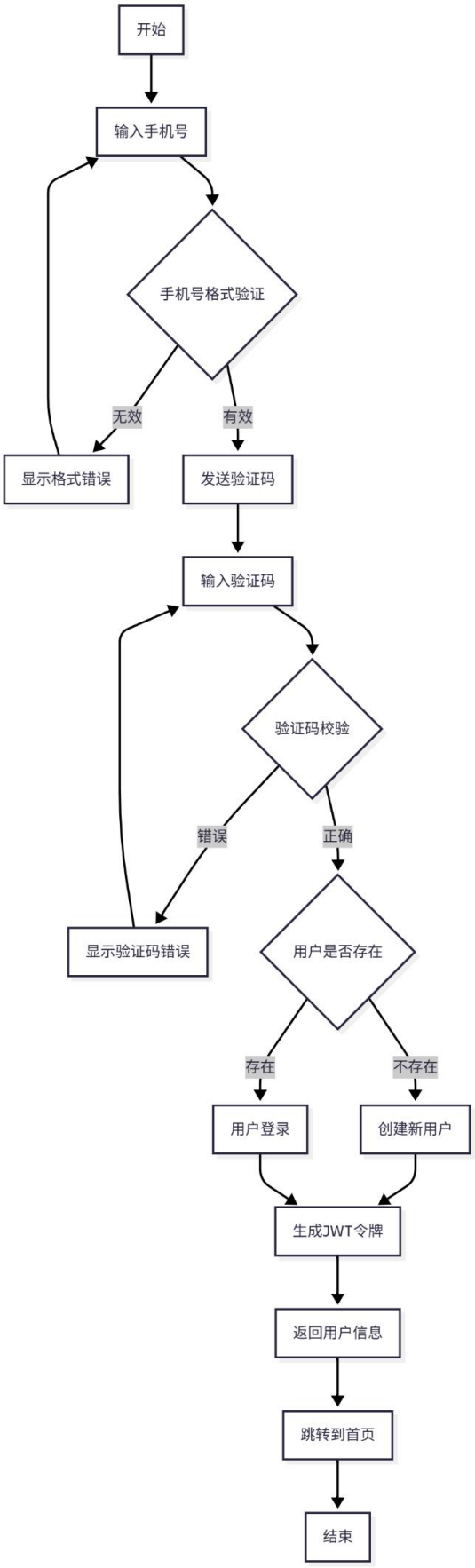
用户登录流程:

1. 验证手机号是否已注册
2. 验证码校验或密码验证
3. 检查账户状态（是否被封禁）
4. 更新最后登录时间和设备信息
5. 返回用户信息和访问令牌

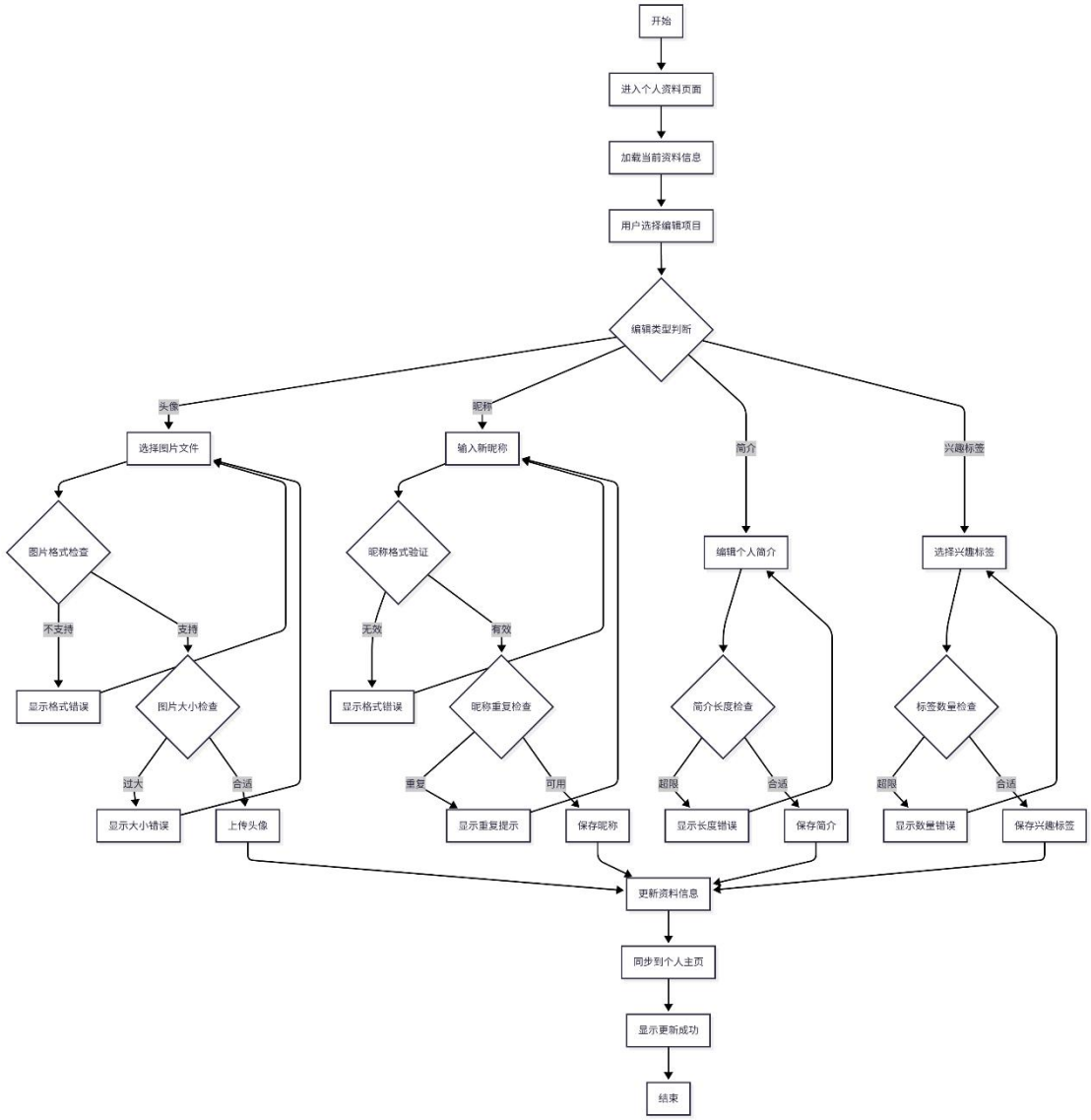
3.1.1.4 异常处理机制

- **手机号已存在**：返回错误码 40001，提示"手机号已注册"
- **验证码错误/过期**：返回错误码 40002，提示"验证码无效"
- **账户被封禁**：返回错误码 40003，提示"账户已被封禁"
- **令牌过期**：返回错误码 40101，客户端自动刷新令牌
- **网络异常**：实现重试机制，最多重试 3 次

3.1.1.5 用户注册登录活动图


开始
输入手机号
手机号格式验证
无效
有效
显示格式错误
发送验证码
输入验证码
验证码校验
错误
正确
显示验证码错误
用户是否存在
存在
不存在
用户登录
创建新用户
生成JWT令牌
返回用户信息
跳转到首页
结束

3.1.1.6 个人资料管理活动图



3.1.2 模块设计说明

3.1.2.2 数据存储设计

- **主数据库**：MySQL 存储用户基本信息、认证信息
- **缓存层**：Redis 存储验证码、会话信息、用户权限
- **文件存储**：OSS 存储用户头像等静态资源

3.1.2.3 安全设计

- **密码加密**：加盐后通过算法加密存储
- **JWT 令牌**：访问令牌有效期 2 小时

- **接口限流**：验证码发送限制每分钟 1 次，登录尝试限制每分钟 5 次
- **数据脱敏**：日志中手机号中间 4 位用*替代

3.2 视频模块

3.2.1 功能模块描述及流程

3.2.1.1 功能概述

视频模块是知音 App 的核心内容模块，负责视频的导入、存储、播放、管理和与文档内容的混合发布功能。

3.2.1.2 功能输入输出要求

FR-C-01: 视频导入

- **输入要求**:
    - 视频文件：支持 MP4、MOV、AVI 格式，大小 < 500MB
    - 视频分辨率：支持 720p、1080p、4K（3840×2160）
    - 视频时长：最短 30 秒，最长 60 分钟
    - 帧率：支持 24fps、30fps、60fps
- **输出要求**:
    - 视频 ID：String 类型，全局唯一标识
    - 视频 URL：String 类型，CDN 加速地址
    - 视频元数据：包含时长、分辨率、文件大小、编码格式

FR-C-03: 混合发布流程

- **输入要求**:
    - 视频 ID：String 类型，已上传视频的标识
    - 标题：String 类型，长度 2-100 字符
    - 描述：String 类型，长度 < 2000 字符
    - 话题标签：Array 类型，最多 5 个标签
    - 封面图：图片文件，JPG/PNG 格式，大小 < 2MB
    - 隐私设置：Enum 类型（公开、仅粉丝、私密）
- **输出要求**:

- 混合内容 ID：String 类型，发布内容的唯一标识
- 发布状态：Enum 类型（发布成功、审核中、发布失败）

视频播放功能

- **输入要求**：
  - 视频 ID：String 类型
  - 播放质量：Enum 类型（自动、720p、1080p、4K）
  - 播放位置：Integer 类型，秒为单位
- **输出要求**：
  - 视频流：HLS 或 DASH 格式
  - 播放统计：播放次数、完播率、平均观看时长

3.2.1.3 业务处理逻辑

视频上传处理流程：

1. 客户端选择本地视频文件
2. 文件格式和大小验证
3. 生成上传凭证和分片上传策略
4. 分片上传到对象存储服务
5. 服务端合并分片并触发转码任务
6. 多分辨率转码和 CDN 分发
7. 更新视频状态为"可用"

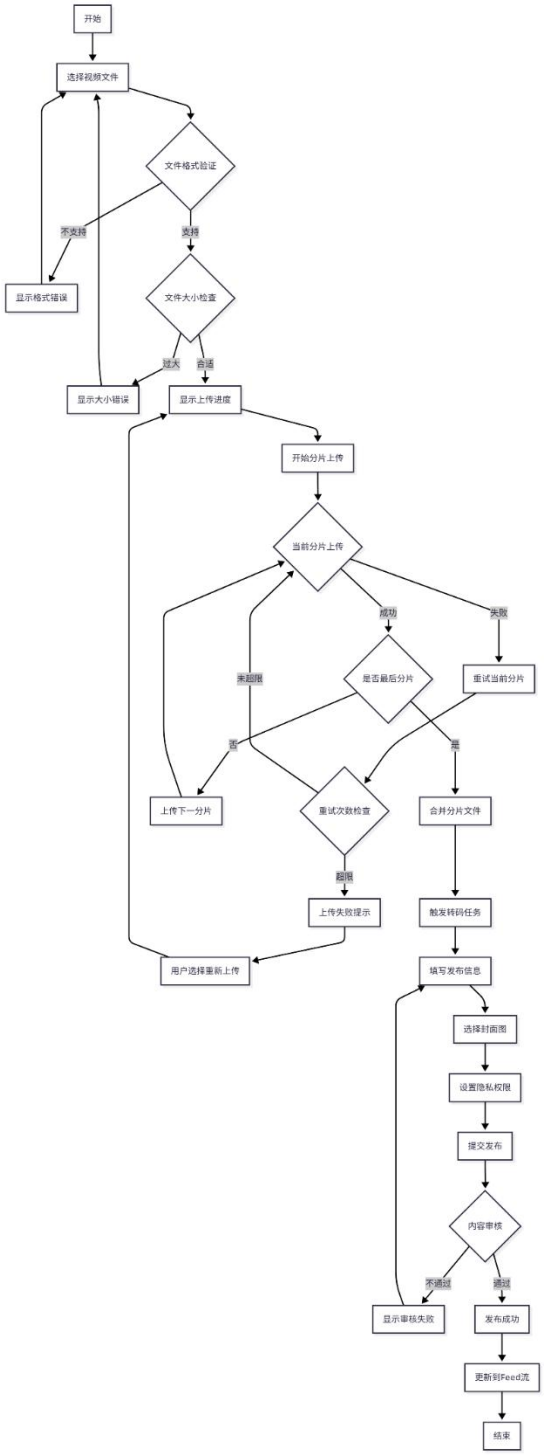
视频播放处理流程：

1. 根据视频 ID 查询视频元数据
2. 检查用户访问权限
3. 根据网络状况和设备性能选择最佳清晰度
4. 返回视频流地址和播放参数
5. 记录播放行为数据（开始时间、暂停、跳转等）
6. 更新播放统计数据

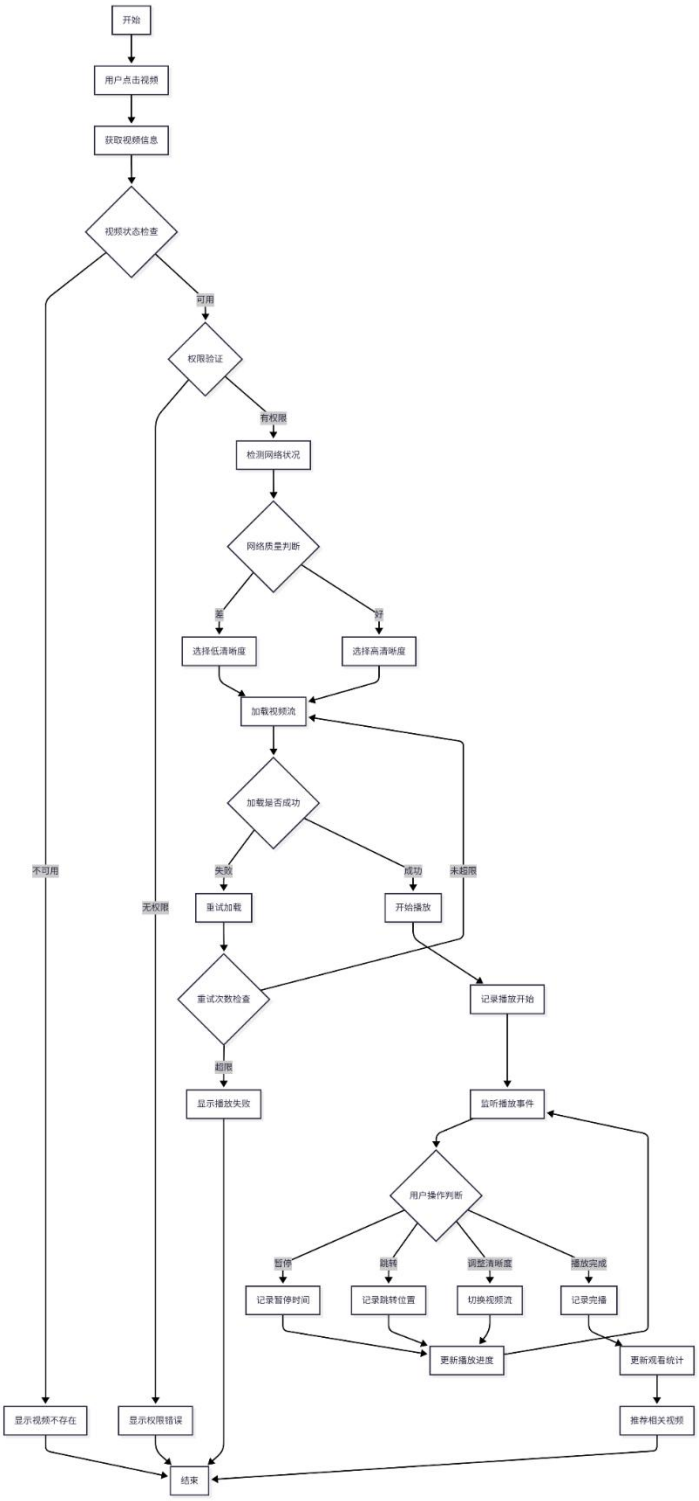
3.2.1.4 异常处理机制

- **文件格式不支持**：返回错误码 50001，提示"不支持的视频格式"
- **文件过大**：返回错误码 50002，提示"视频文件过大，请压缩后上传"
- **分片上传失败**：返回错误码 50003，支持单个分片重试机制（最多 3 次）
- **转码失败**：返回错误码 50004，自动重试转码任务
- **播放失败**：返回错误码 50005，降级到低清晰度播放
- **网络异常**：实现自适应码率和预加载机制

3.2.1.5 视频上传发布活动图



3.2.1.6 视频播放观看活动图



3.2.2 模块设计说明

3.2.2.1 架构设计

视频模块采用微服务架构:

- **上传服务**：处理视频文件上传

- **转码服务**：异步视频转码，生成多分辨率版本
- **播放服务**：视频流分发，播放统计收集

3.2.2.2 存储设计

- **对象存储**：存储原始视频和转码后文件
- **数据库**：MySQL 存储视频元数据、发布信息
- **缓存**：Redis 缓存热门视频信息和播放统计

3.2.2.3 转码策略

- **格式转换**：统一转换为 H.264 编码的 MP4 格式
- **缩略图生成**：自动提取关键帧作为封面候选

3.2.2.4 播放优化

- **缓存策略**：本地缓存已观看视频片段
- **网络自适应**：根据网速自动调整清晰度
- **播放统计**：实时收集播放数据用于推荐算法

3.3 社交互动模块

3.3.1 功能模块描述及流程

3.3.1.1 功能概述

社交互动模块负责用户之间的社交功能，包括基础互动（点赞、评论、分享、收藏）、私信通讯、关注关系管理和个人主页展示等功能。

3.3.1.2 功能输入输出要求

FR-S-01: 基础互动

- **输入要求**：
    - 点赞操作：内容 ID、用户 ID
    - 评论内容：String 类型，长度 1-500 字符，支持@用户和表情符号
    - 分享设置：分享平台（Enum 类型：微信、QQ、微博、复制链接）
    - 收藏分类：收藏夹 ID（Long 类型，可选）
- **输出要求**：
    - 操作结果：Boolean 类型，操作成功/失败状态

    - 统计更新：点赞数、评论数、分享数、收藏数实时更新
    - 通知触发：向内容作者发送互动通知
- **性能指标**：互动响应时间 < 200ms，通知推送延迟 < 3s

FR-S-02: 私信功能

- **输入要求**：
    - 接收用户 ID：String 类型
    - 消息内容：String 类型，长度 < 1000 字符
    - 消息类型：Enum 类型（文字、图片、语音、视频分享）
    - 附件文件：图片 < 10MB，语音 < 60s
- **输出要求**：
    - 消息 ID：String 类型，消息唯一标识
    - 发送状态：Enum 类型（发送中、已发送、已读、发送失败）
    - 会话更新：最新消息时间、未读消息数

FR-S-03: 个人主页

- **输入要求**：
    - 用户 ID：Long 类型，目标用户标识
    - 内容类型筛选：Enum 类型（全部、视频、文档、收藏）
    - 分页参数：pageSize（Integer，默认 20），pageNum（Integer）
- **输出要求**：
    - 用户基本信息：头像、昵称、简介、认证状态
    - 社交数据：粉丝数、关注数、获赞数、作品数
    - 内容列表：按时间倒序排列的用户作品

3.3.1.3 业务处理逻辑

点赞评论处理流程：

1. 验证用户身份和权限
2. 检查内容是否存在且可访问
3. 防重复点赞检查（基于用户 ID+内容 ID）

4. 更新点赞/评论记录到数据库
5. 异步更新内容统计数据
6. 触发通知推送给内容作者
7. 更新用户行为数据用于推荐算法

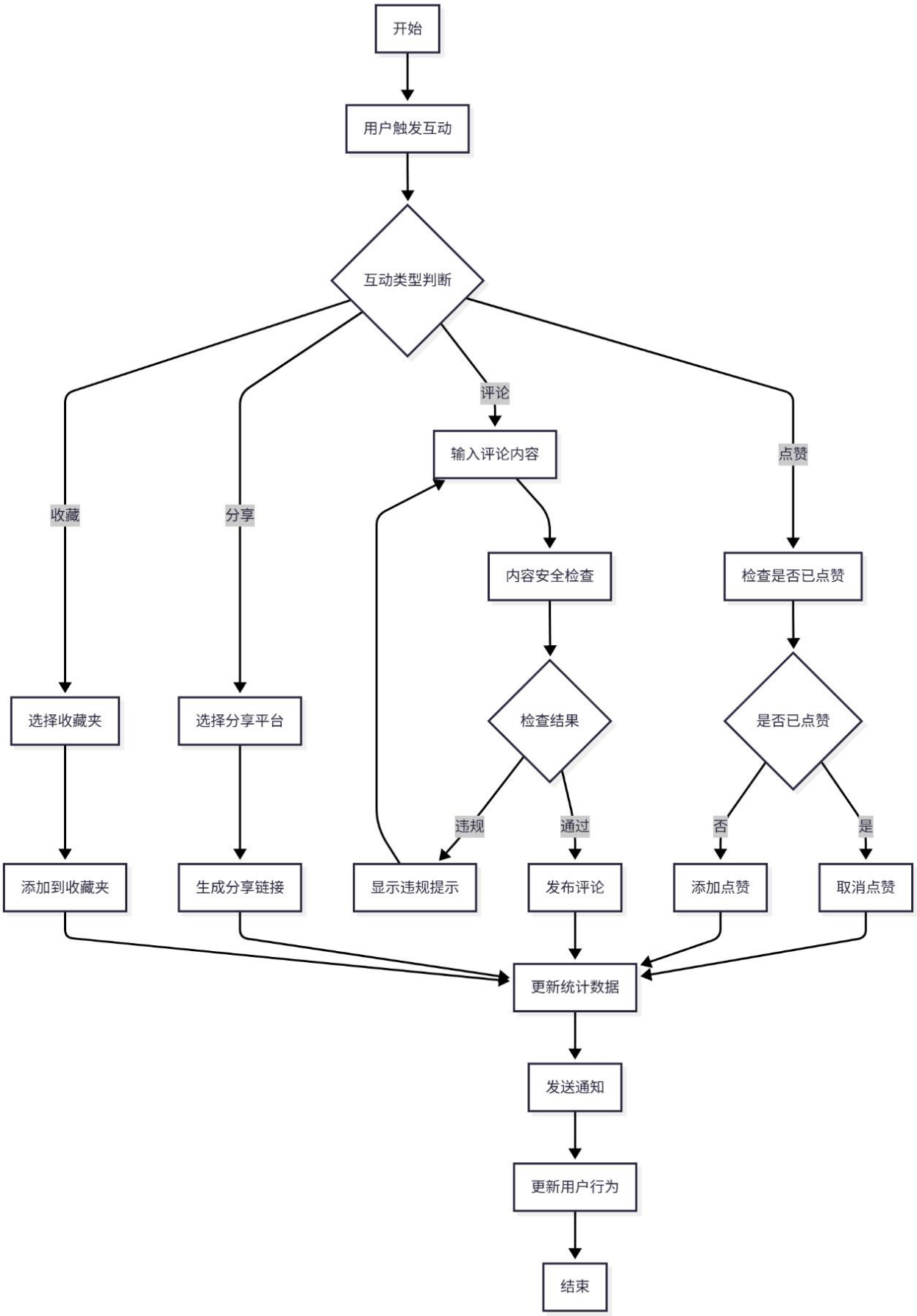
私信发送处理流程:

1. 验证发送者和接收者用户状态
2. 检查是否被对方拉黑或设置免打扰
3. 内容安全检查和敏感词过滤
4. 创建消息记录并分配消息 ID
5. 推送消息到接收者设备
6. 更新会话列表和未读消息计数
7. 记录消息发送状态

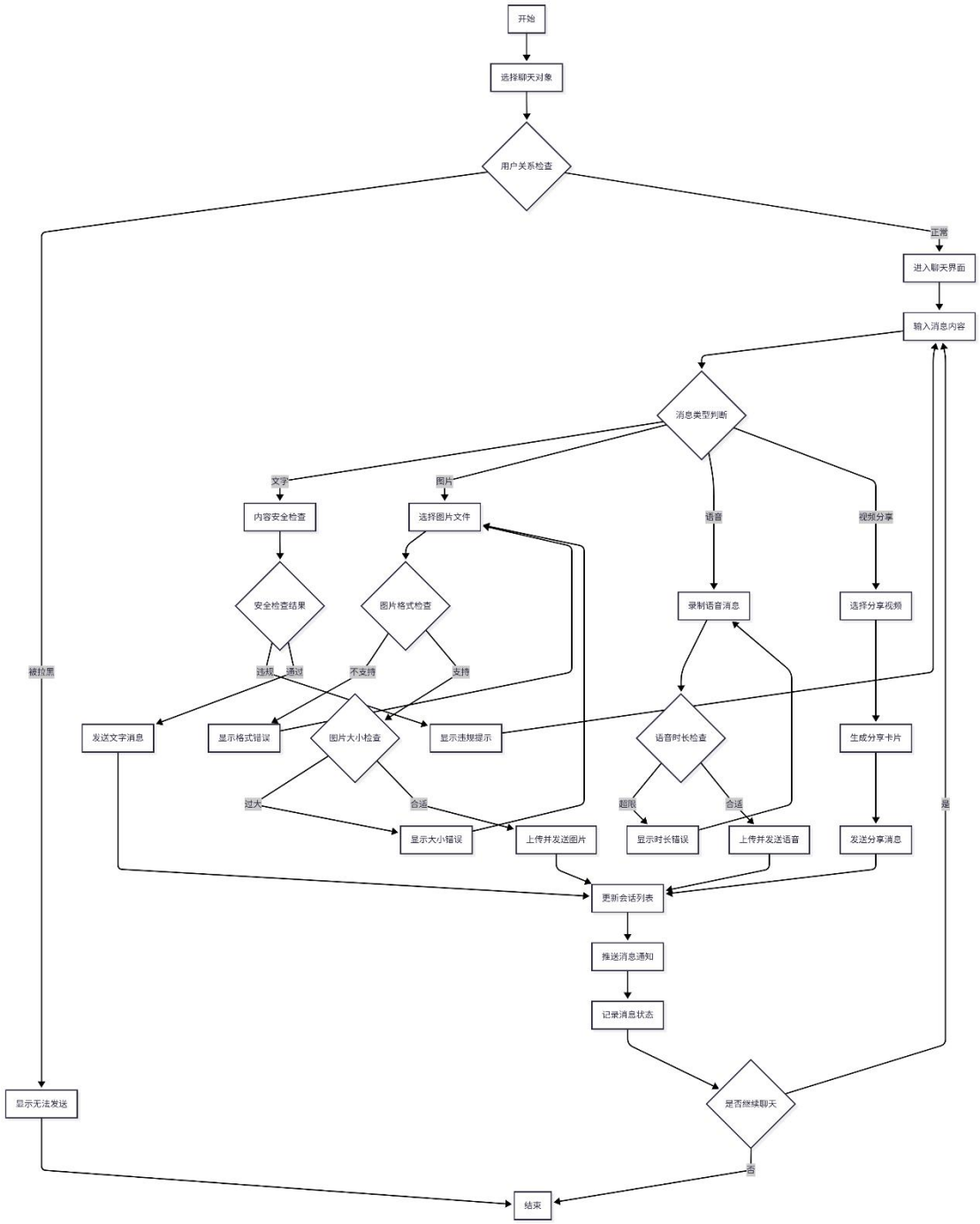
3.3.1.4 异常处理机制

- **重复点赞**：返回错误码 60001，提示"已经点过赞了"
- **评论内容违规**：返回错误码 60002，提示"评论内容包含敏感词"
- **用户被拉黑**：返回错误码 60003，提示"无法向该用户发送消息"
- **消息发送失败**：返回错误码 60004，支持消息重发机制
- **网络异常**：本地缓存操作，网络恢复后同步
- **频繁操作**：实现防刷机制，限制操作频率

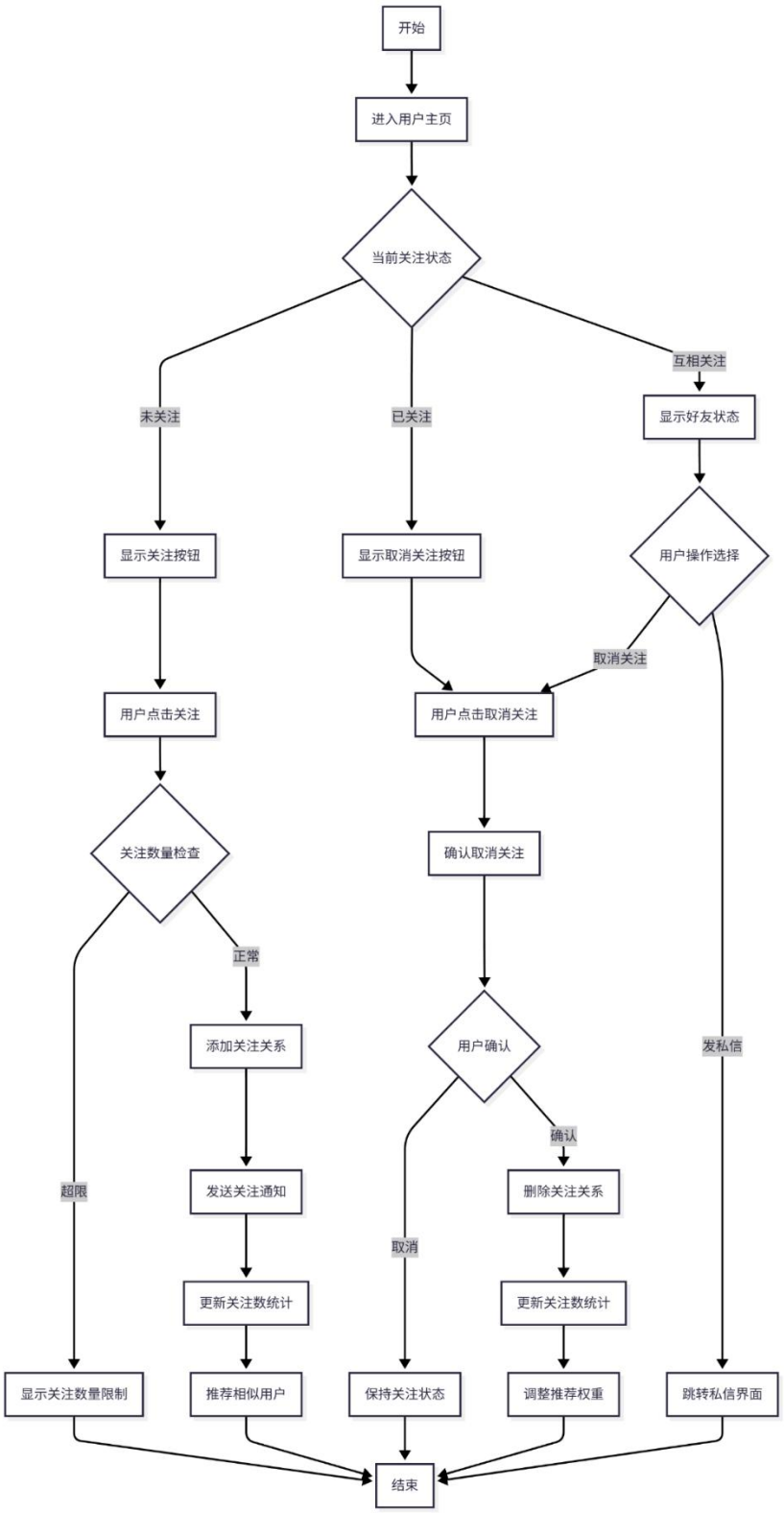
3.3.1.5 社交互动活动图


开始
用户触发互动
互动类型判断
收藏
分享
评论
点赞
选择收藏夹
选择分享平台
输入评论内容
内容安全检查
检查结果
违规
通过
显示违规提示
发布评论
检查是否已点赞
是否已点赞
否
是
添加点赞
取消点赞
添加到收藏夹
生成分享链接
更新统计数据
发送通知
更新用户行为
结束

3.3.1.6 私信发送活动图



3.3.1.7 关注关系管理活动图



3.3.2 模块设计说明

3.3.2.1 架构设计

社交互动模块采用事件驱动架构:

- **互动服务**：处理点赞、评论、分享、收藏操作
- **消息服务**：处理私信发送、接收、推送
- **关系服务**：管理用户关注、粉丝关系
- **通知服务**：统一处理各类通知推送

3.3.2.2 数据存储设计

- **关系数据库**：MySQL 存储用户关系、评论、私信等结构化数据
- **缓存层**：Redis 缓存热门内容的互动数据、用户关系
- **消息队列**：Kafka 处理异步通知推送和数据统计
- **搜索引擎**：Elasticsearch 支持评论内容搜索

3.3.2.3 性能优化策略

- **读写分离**：互动数据读多写少，采用主从分离
- **异步处理**：统计数据更新和通知推送异步执行
- **缓存预热**：热门内容的互动数据预加载到缓存
- **防刷机制**：基于用户 ID 和 IP 的频率限制

3.3.2.4 安全设计

- **内容审核**：评论内容实时敏感词检测
- **防刷保护**：同一用户对同一内容的操作频率限制
- **隐私保护**：私信内容加密存储和传输
- **黑名单机制**：支持用户拉黑和举报功能

3.4 笔记模块

3.4.1 功能模块描述及流程

3.4.1.1 功能概述

笔记模块是知音 App 的核心学习功能，支持三种不同类型的笔记：视频作者上传的附件笔记、用户创建的视频关联个人笔记和完全独立的自由笔记，为用户提供完整的知

识记录和管理体验。

3.4.1.2 笔记类型定义

类型一：视频附件笔记（Author Notes）

- **定义**：由视频作者创建并上传的配套文档，与视频内容强绑定
- **存储方式**：存储在 notes 表中，note_type='author_note'，通过 video_id 关联
- **访问权限**：与视频权限一致，公开视频的附件笔记对所有用户可见
- **内容格式**：支持 Markdown 格式，包含文本、图片、代码块、数学公式

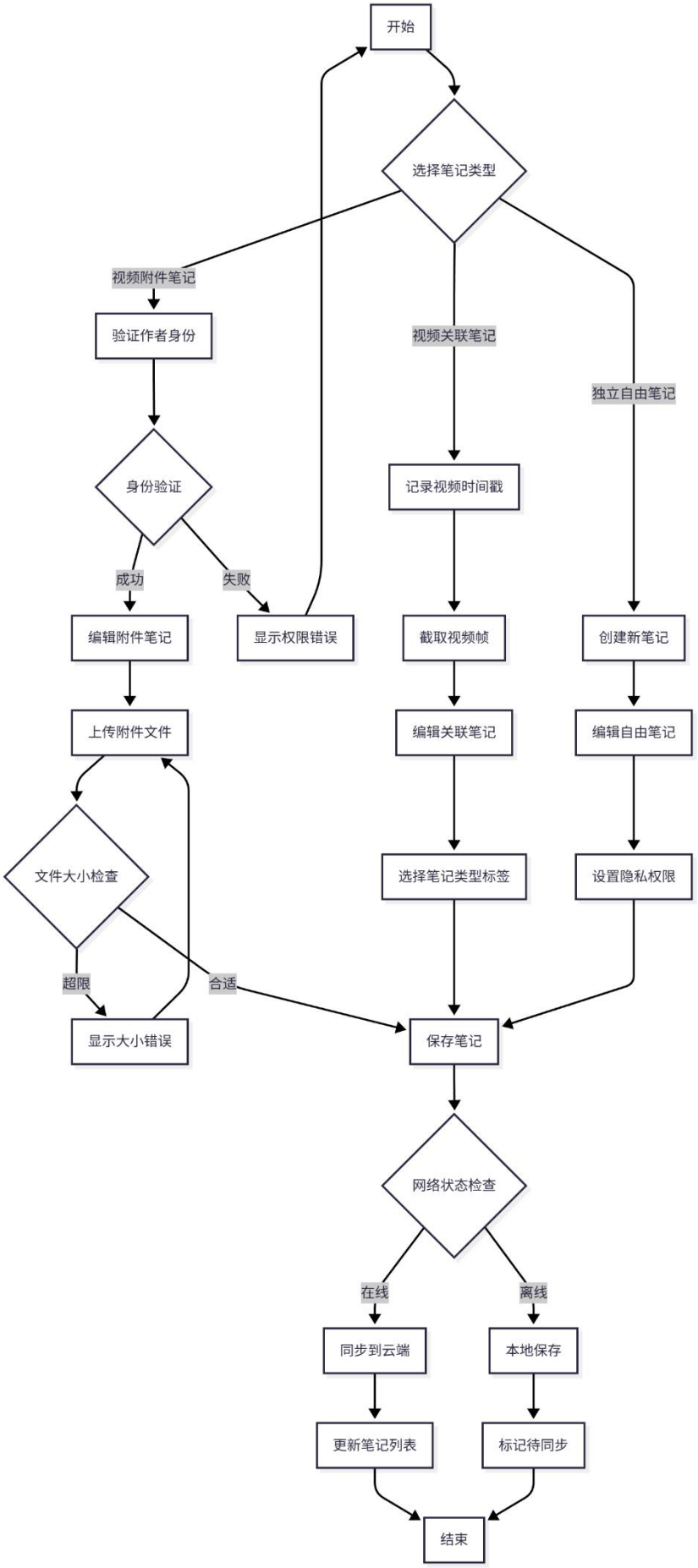
类型二：视频关联个人笔记（User Video Notes）

- **定义**：用户在观看视频时创建的个人学习笔记，与特定视频和时间点关联
- **存储方式**：存储在 notes 表中，note_type='user_video_note'，包含 user_id、video_id、timestamp
- **访问权限**：仅笔记创建者可见，完全私有
- **内容格式**：支持富文本编辑，可插入截图、时间戳标记

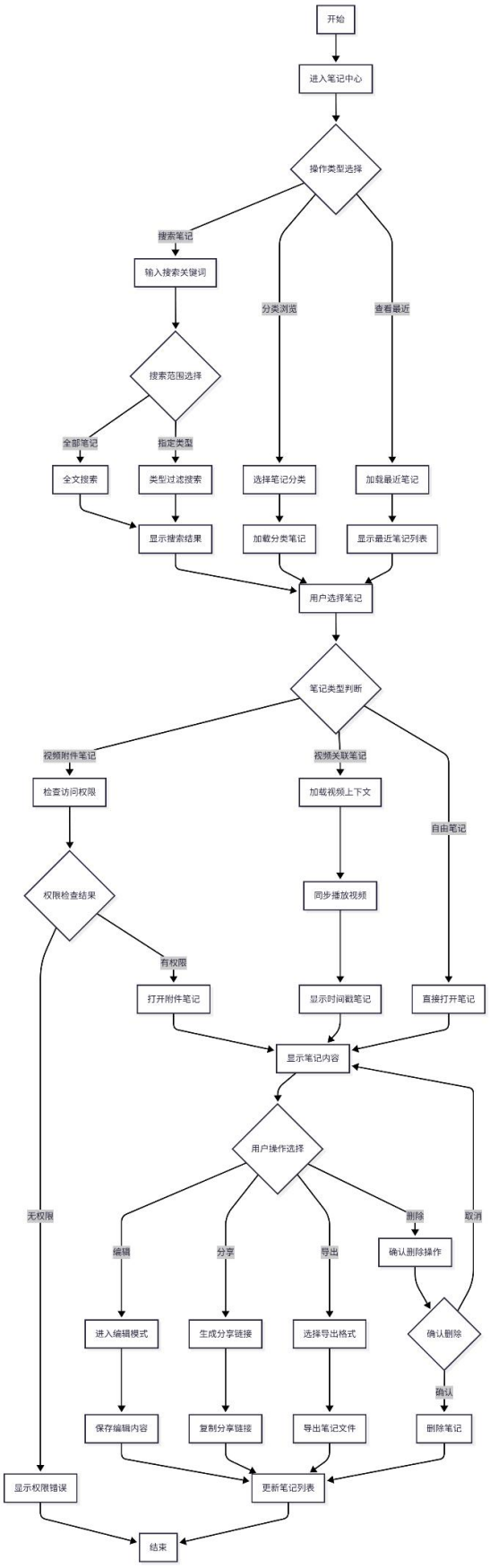
类型三：独立自由笔记（Free Notes）

- **定义**：用户创建的完全独立笔记，不与任何视频关联
- **存储方式**：存储在 notes 表中，note_type='free_note'，仅关联 user_id，video_id 为 NULL
- **访问权限**：默认私有，用户可选择公开分享
- **内容格式**：支持完整的富文本编辑和文件附件

3.4.1.3 笔记创建管理活动图


开始
选择笔记类型
视频附件笔记
视频关联笔记
独立自由笔记
验证作者身份
身份验证
记录视频时间戳
成功
失败
编辑附件笔记
显示权限错误
截取视频帧
创建新笔记
上传附件文件
编辑关联笔记
编辑自由笔记
文件大小检查
选择笔记类型标签
设置隐私权限
超限
合适
显示大小错误
保存笔记
网络状态检查
在线
离线
同步到云端
本地保存
更新笔记列表
标记待同步
结束

3.4.1.4 笔记搜索查看活动图



3.4.2 模块设计说明

3.4.2.1 架构设计

笔记模块采用分层存储架构：

- **编辑服务**：富文本编辑器，支持实时保存
- **存储服务**：分类存储不同类型笔记
- **搜索服务**：全文搜索，标签分类，智能推荐

3.4.2.2 数据存储设计

统一笔记存储架构：

为了简化数据管理和提高开发效率，采用统一的笔记存储架构：

核心表结构：

- **主表**：notes 表，存储所有类型笔记的元数据
    - note_id (主键)
    - note_type (ENUM: 'author_note', 'user_video_note', 'free_note')
    - user_id (创建者 ID)
    - video_id (关联视频 ID，可为 NULL)
    - timestamp (视频时间戳，仅视频关联笔记使用)
    - title (笔记标题)
    - privacy_level (隐私级别)
    - created_time, updated_time
- **内容表**：note_contents 表，存储笔记具体内容
    - note_id (外键关联 notes 表)
    - content_type (ENUM: 'markdown', 'rich_text', 'html')
    - content_data (TEXT/JSON 格式存储内容)
    - version (版本号，支持历史版本)
- **附件表**：note_attachments 表，存储文件附件
    - attachment_id (主键)
    - note_id (外键关联 notes 表)

    - file_name, file_path, file_size, file_type
- **标签表**：note_tags 表，存储标签关联
    - note_id (外键)
    - tag_name (标签名称)

索引策略优化：

- **主索引**：note_id (主键)
- **类型索引**：note_type + user_id (按类型查询用户笔记)
- **视频关联索引**：video_id + note_type (查询视频相关笔记)
- **时间戳索引**：user_id + video_id + timestamp (视频学习笔记定位)
- **全文索引**：title, content_data (支持全文搜索)
- **时间索引**：created_time, updated_time (按时间排序)

3.4.2.3 搜索和组织

- **全文搜索**：基于 Elasticsearch 的笔记内容搜索
- **智能分类**：基于内容的自动标签推荐
- **导出功能**：支持 PDF、Markdown 格式导出

3.5 内容消费与推荐模块

3.5.1 功能模块描述及流程

3.5.1.1 功能概述

内容消费与推荐模块是知音 App 的核心引擎，负责为用户提供个性化的视频推荐、智能内容分发和优质的观看体验，通过机器学习算法分析用户行为，实现精准的内容匹配和推送。

3.5.1.2 功能输入输出要求

个性化推荐功能

- **输入要求**：
    - 用户 ID：String 类型，已登录用户标识
    - 用户画像：JSON 格式，包含兴趣标签、学习偏好、活跃时间
    - 历史行为：Array 类型，观看记录、点赞、收藏、分享行为

    - 上下文信息：设备类型、网络状况、当前时间、地理位置

- **输出要求**：
    - 推荐视频列表：Array 类型，包含视频 ID、标题、封面、时长
    - 推荐理由：String 类型，解释推荐原因
    - 置信度分数：Float 类型，范围 0.0-1.0，推荐准确度预估
    - 推荐策略标识：Enum 类型（协同过滤、内容相似、热门推荐、新内容发现）

内容搜索功能

- **输入要求**：
    - 搜索关键词：String 类型，长度 1-100 字符，支持中英文
    - 搜索类型：Enum 类型（视频、用户、笔记、综合）
    - 过滤条件：JSON 格式，时长、分类、发布时间、难度等级
    - 排序方式：Enum 类型（相关度、时间、热度、评分）
- **输出要求**：
    - 搜索结果列表：Array 类型，分页返回，每页 20 条
    - 搜索建议：Array 类型，相关搜索词推荐
    - 结果统计：总数量、各类型分布、搜索耗时
    - 高亮信息：匹配关键词的高亮位置
- **性能指标**：搜索响应时间 < 100ms，搜索准确率 > 90%

内容分类浏览功能

- **输入要求**：
    - 分类 ID：Long 类型，内容分类标识
    - 子分类筛选：Array 类型，多级分类过滤
    - 排序规则：Enum 类型（最新、最热、评分、推荐）
    - 分页参数：页码和每页数量，支持无限滚动加载
- **输出要求**：
    - 分类视频列表：Array 类型，包含完整视频信息

- 分类统计：视频总数、平均评分、更新频率
- 推荐标签：相关分类和热门标签

3.5.1.3 业务处理逻辑

个性化推荐算法流程：

1. 收集用户多维度行为数据（观看、互动、搜索、停留时间）
2. 构建用户兴趣画像和内容特征向量
3. 应用协同过滤算法发现相似用户和内容
4. 结合内容相似度和热度进行综合排序
5. 应用多样性策略避免信息茧房
6. 实时调整推荐权重和策略参数
7. 记录推荐结果和用户反馈用于模型优化

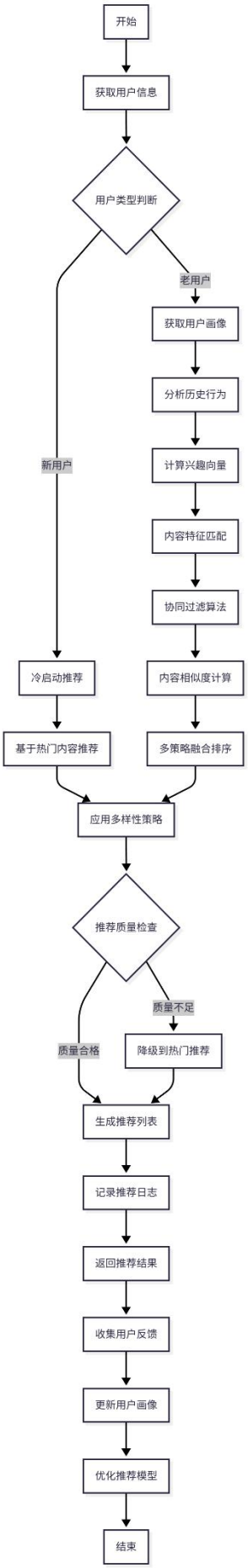
内容冷启动处理流程：

1. 新用户：基于注册信息和初始兴趣选择推荐热门内容
2. 新视频：基于内容标签和作者历史表现进行初始推广
3. 收集用户对新内容的反馈和互动数据
4. 快速调整新内容的推荐权重和分发策略
5. 逐步建立新用户画像和新内容特征

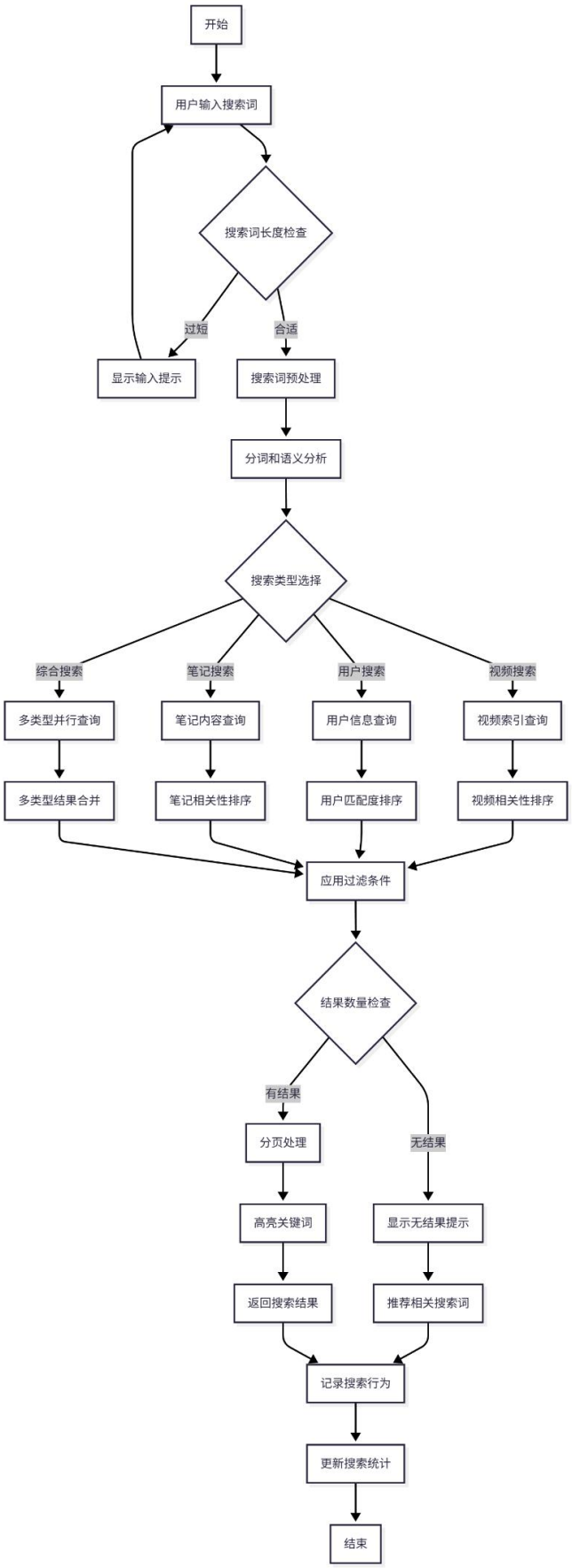
3.5.1.4 异常处理机制

- **推荐服务异常**：返回错误码 80001，降级到热门内容推荐
- **搜索服务超时**：返回错误码 80002，提供缓存结果或搜索建议
- **用户画像缺失**：返回错误码 80003，使用默认推荐策略
- **内容库为空**：返回错误码 80004，推荐平台精选内容
- **网络延迟过高**：启用本地缓存，预加载推荐内容
- **算法模型异常**：自动切换到备用推荐策略

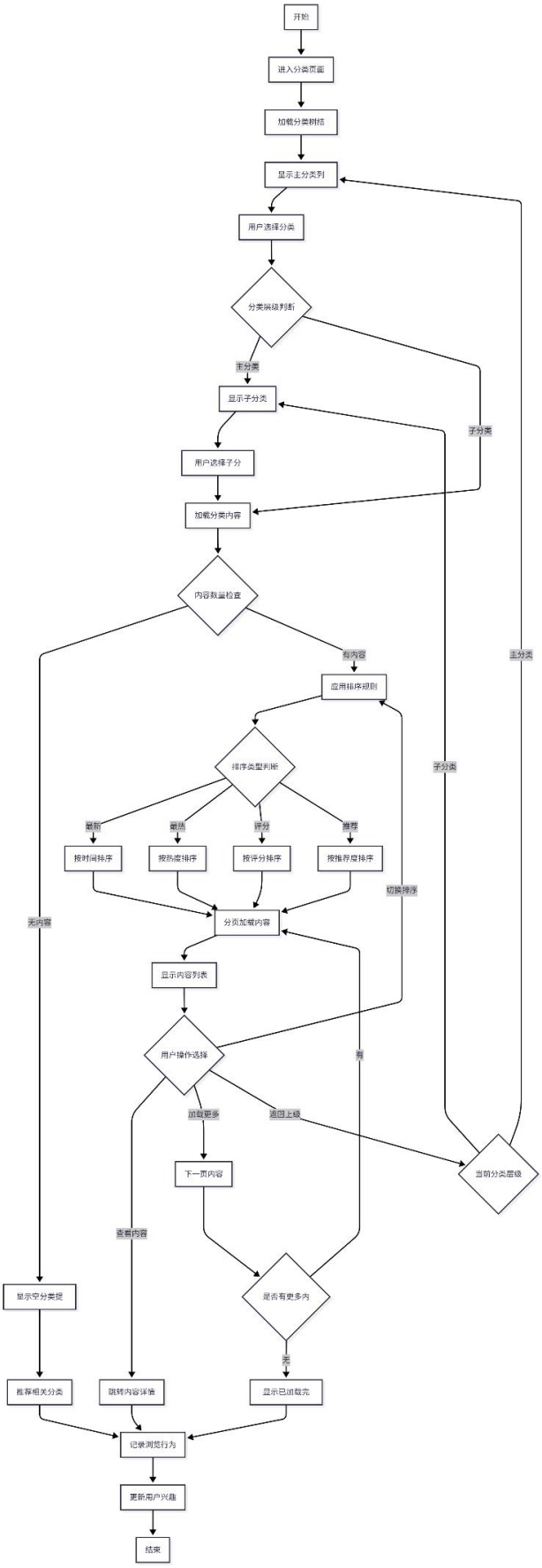
3.5.1.5 内容推荐流程活动图


开始
获取用户信息
用户类型判断
新用户
老用户
冷启动推荐
基于热门内容推荐
获取用户画像
分析历史行为
计算兴趣向量
内容特征匹配
协同过滤算法
内容相似度计算
多策略融合排序
应用多样性策略
推荐质量检查
质量合格
质量不足
降级到热门推荐
生成推荐列表
记录推荐日志
返回推荐结果
收集用户反馈
更新用户画像
优化推荐模型
结束

3.5.1.6 内容搜索活动图



3.5.1.7 内容分类浏览活动图


开始
进入分类页面
加载分类树结
显示主分类列
用户选择分类
分类层级判断
主分类
显示子分类
子分类
用户选择子分
加载分类内容
内容数量检查
有内容
主分类
应用排序规则
排序类型判断
子分类
最新
最热
评分
推荐
按时间排序
按热度排序
按评分排序
按推荐度排序
切换排序
无内容
分页加载内容
显示内容列表
用户操作选择
有
加载更多
返回上级
当前分类层级
下一页内容
查看内容
是否有更多内
显示空分类提
无
推荐相关分类
跳转内容详情
显示已加载完
记录浏览行为
更新用户兴趣
结束

3.5.2 模块设计说明

3.5.2.1 架构设计

内容消费与推荐模块采用微服务架构:

- **推荐引擎服务**：核心算法服务，支持多种推荐策略
- **用户画像服务**：用户行为分析和兴趣建模
- **内容特征服务**：视频内容分析和特征提取
- **搜索服务**：全文搜索和智能查询处理
- **实时计算服务**：流式数据处理和实时推荐更新

3.5.2.2 数据存储设计

用户行为数据存储:

- **行为日志表**：user_behaviors，存储点击、观看、互动行为
- **用户画像表**：user_profiles，存储兴趣标签和偏好权重
- **会话数据表**：user_sessions，存储实时会话和上下文信息
- **索引策略**：user_id + timestamp 复合索引，behavior_type 分区

内容特征数据存储:

- **视频特征表**：video_features，存储内容标签和特征向量
- **分类层次表**：category_hierarchy，存储多级分类关系
- **热度统计表**：content_popularity，存储实时热度和趋势数据
- **索引策略**：category_id 聚簇索引，feature_vector 向量索引

推荐结果缓存:

- **Redis 缓存**：存储用户推荐列表，TTL 30 分钟
- **本地缓存**：热门内容和分类数据，定期更新
- **CDN 缓存**：推荐接口响应，边缘节点缓存

3.5.2.3 推荐算法设计

协同过滤算法:

- **用户相似度**：基于余弦相似度计算用户兴趣相似性
- **物品相似度**：基于内容特征和用户行为计算视频相似性

- **矩阵分解**：使用 SVD 算法处理稀疏评分矩阵
- **深度学习**：神经网络协同过滤，提升推荐准确性

内容推荐算法：

- **TF-IDF**：基于视频标题、描述、标签的文本相似度
- **Word2Vec**：语义相似度计算，理解内容深层含义
- **图神经网络**：基于用户-内容交互图的推荐
- **多模态融合**：结合文本、图像、音频特征的综合推荐

3.5.2.4 实时推荐优化

在线学习机制：

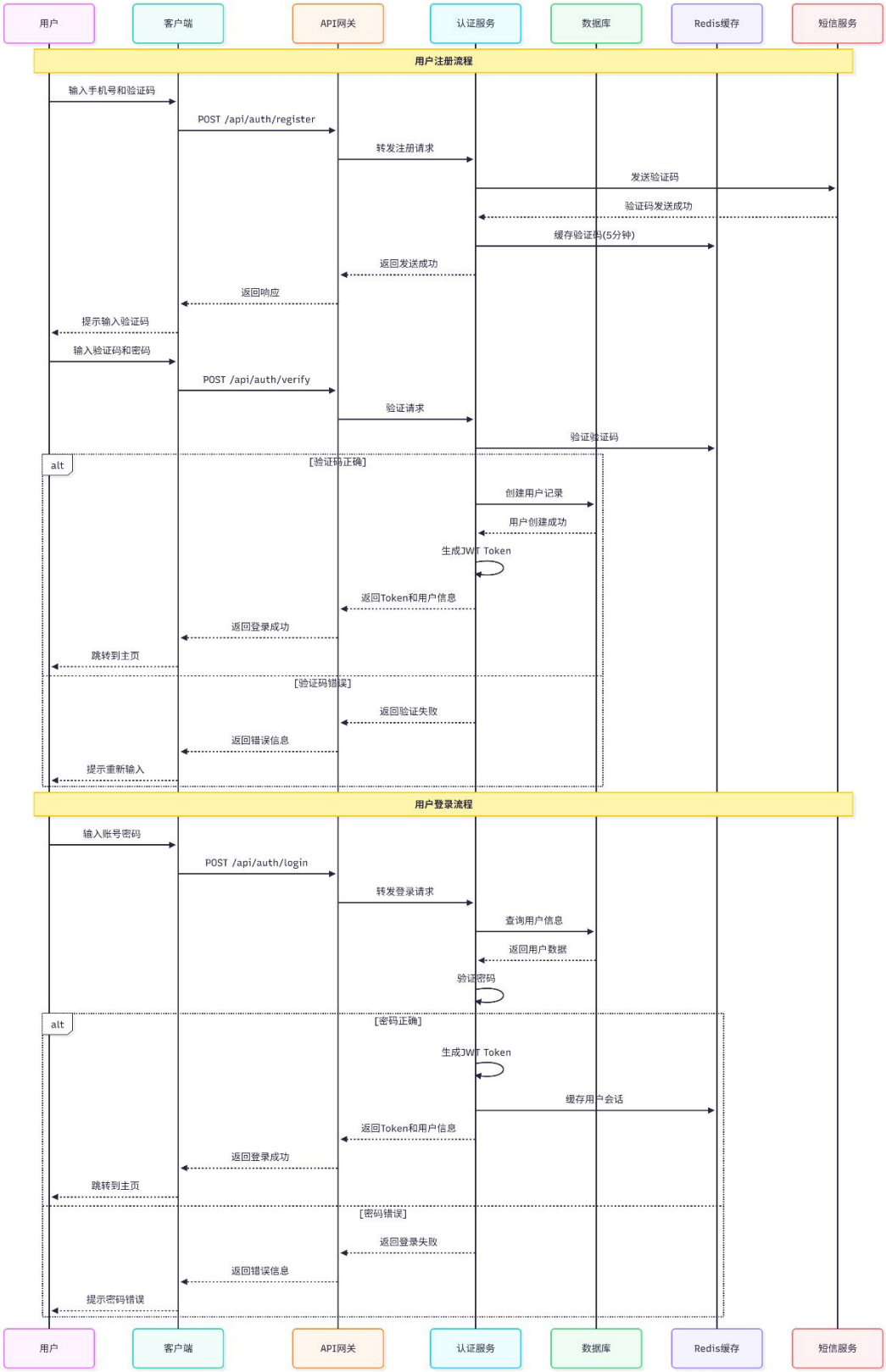
- **增量更新**：实时更新用户画像和内容特征
- **A/B 测试**：多策略并行测试，选择最优推荐算法
- **反馈循环**：用户行为实时反馈到推荐模型
- **冷启动优化**：新用户和新内容的快速适应策略

性能优化策略：

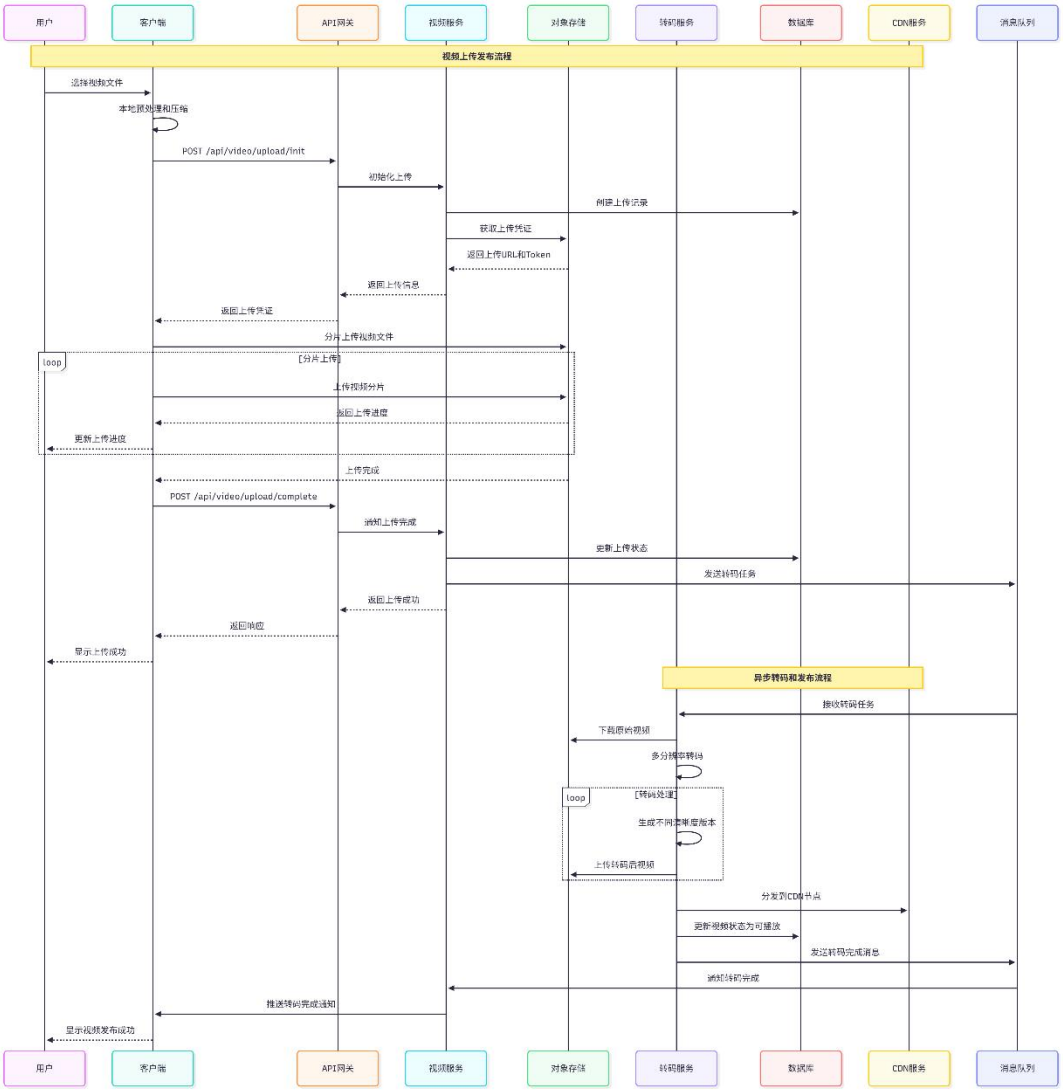
- **分层缓存**：多级缓存策略，减少计算延迟
- **异步处理**：推荐计算和结果返回异步解耦
- **负载均衡**：推荐服务集群，支持高并发访问

3.6 核心业务流程时序图

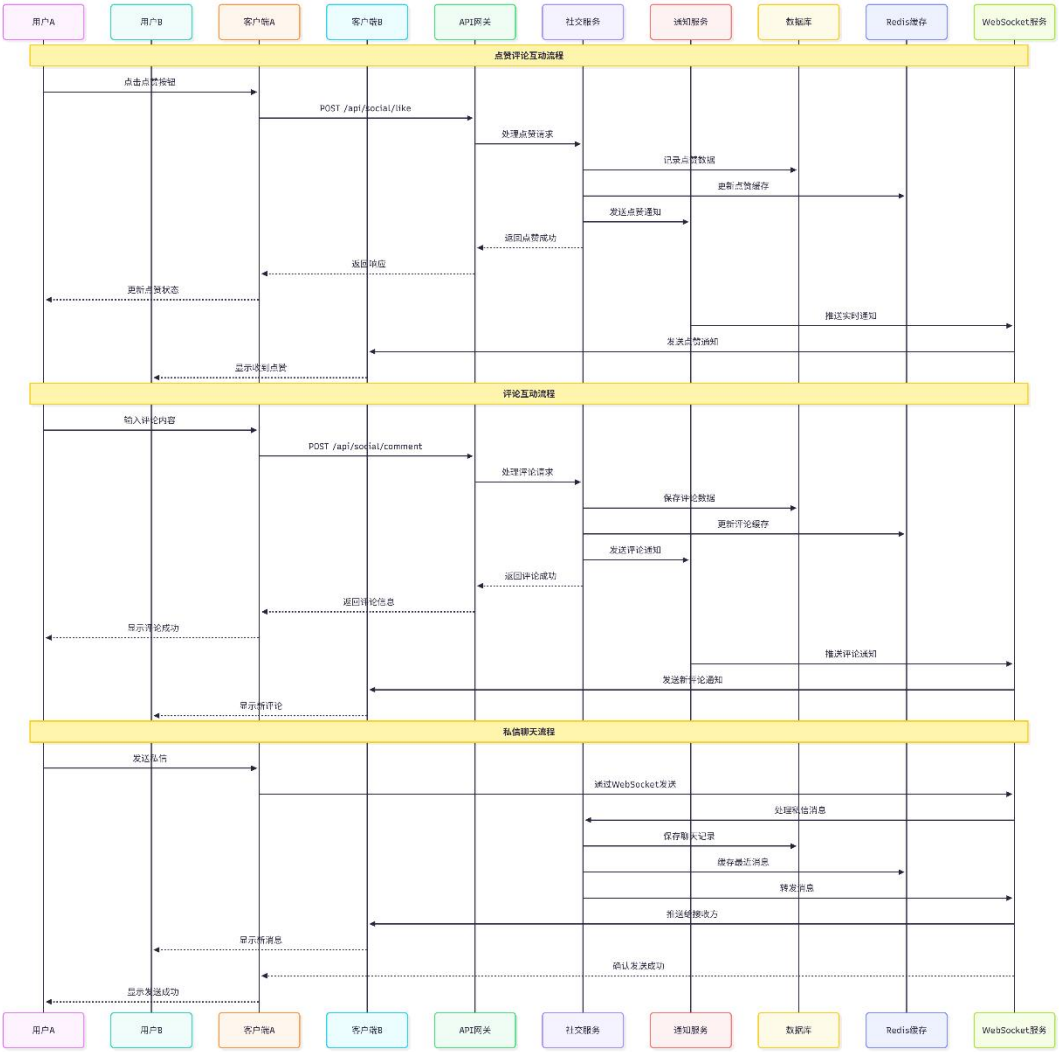
3.6.1 用户注册登录时序图



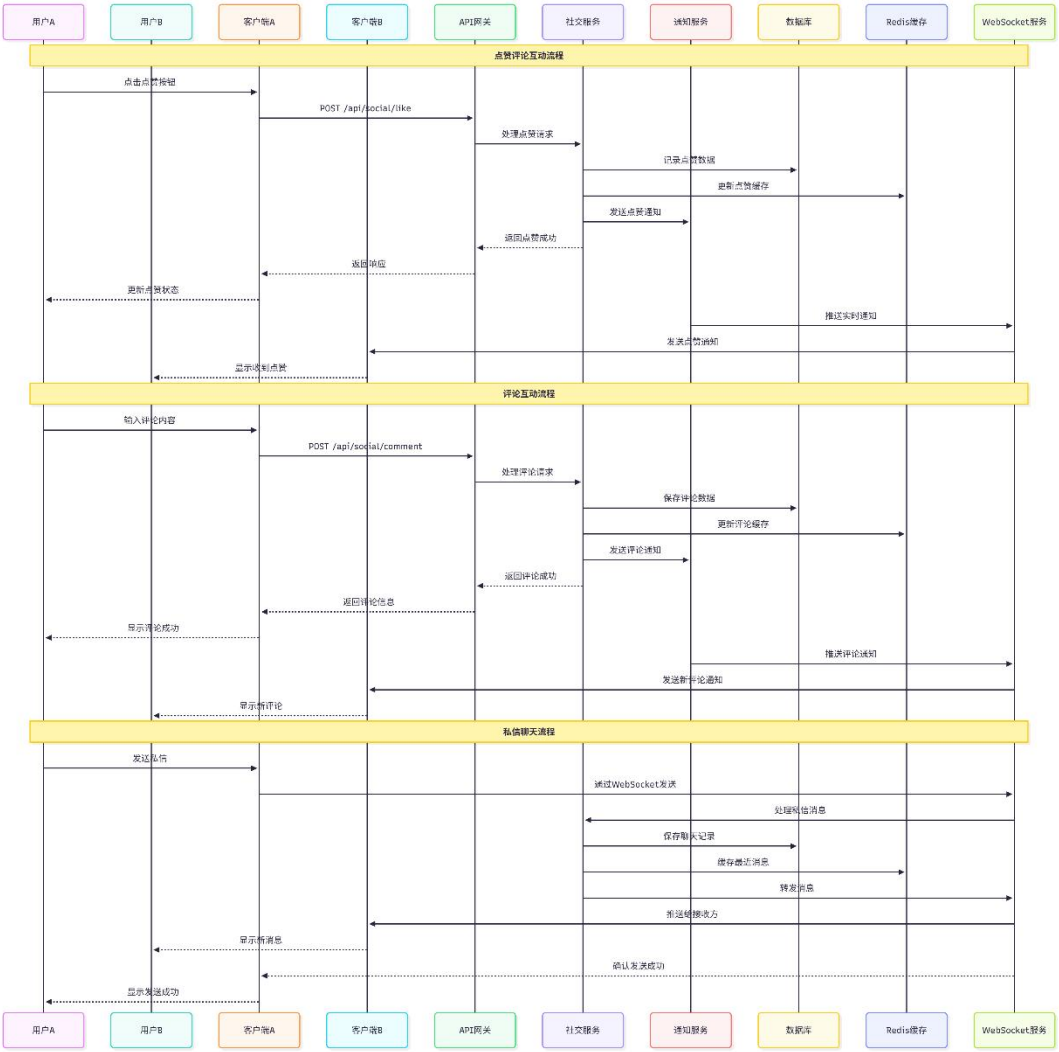
3.6.2 视频上传发布时序图



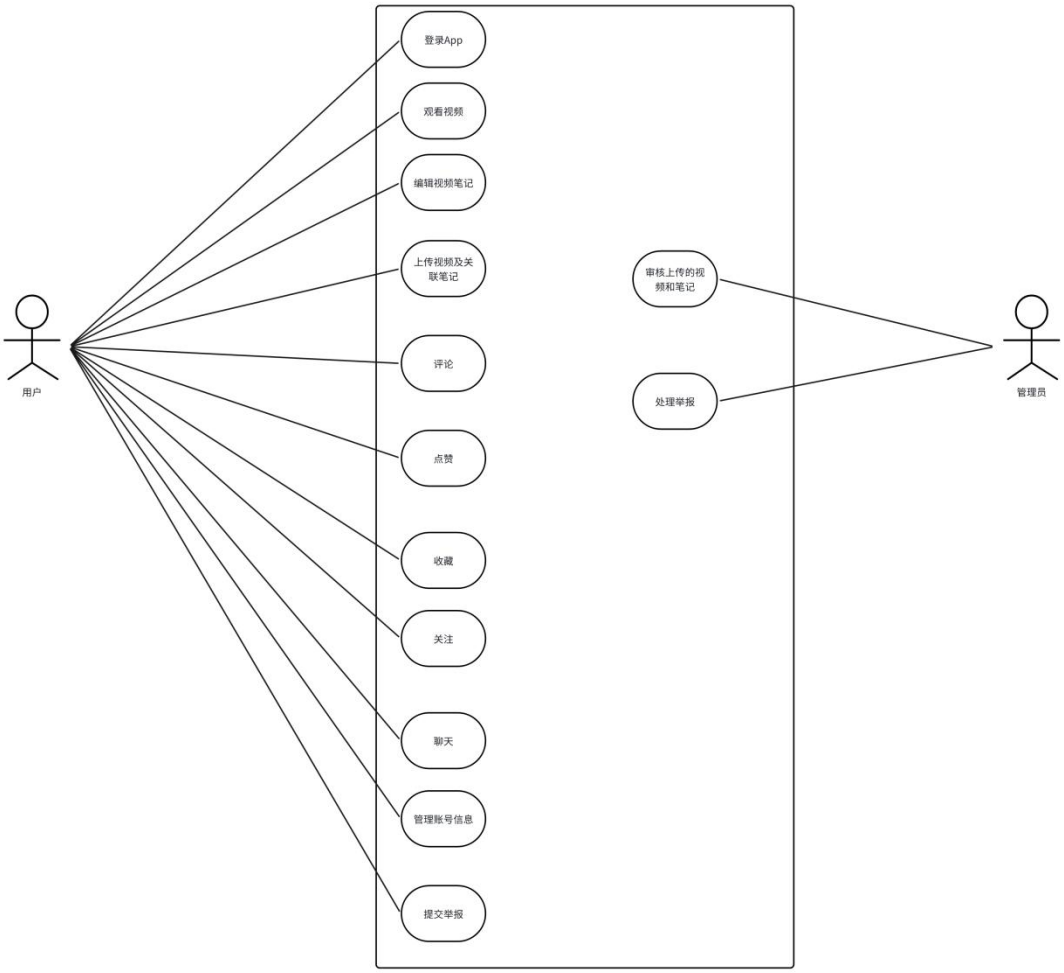
3.6.3 社交互动时序图


用户A
用户B
客户端A
客户端B
API网关
社交服务
通知服务
数据库
Redis缓存
WebSocket服务
点赞评论互动流程
点击点赞按钮
POST /api/social/like
处理点赞请求
记录点赞数据
更新点赞缓存
发送点赞通知
返回点赞成功
返回响应
更新点赞状态
推送实时通知
发送点赞通知
显示收到点赞
评论互动流程
输入评论内容
POST /api/social/comment
处理评论请求
保存评论数据
更新评论缓存
发送评论通知
返回评论成功
返回评论信息
显示评论成功
推送评论通知
发送新评论通知
显示新评论
私信聊天流程
发送私信
通过WebSocket发送
处理私信消息
保存聊天记录
缓存最近消息
转发消息
推送给接收方
显示新消息
确认发送成功
显示发送成功

3.6.4 个性化推荐时序图

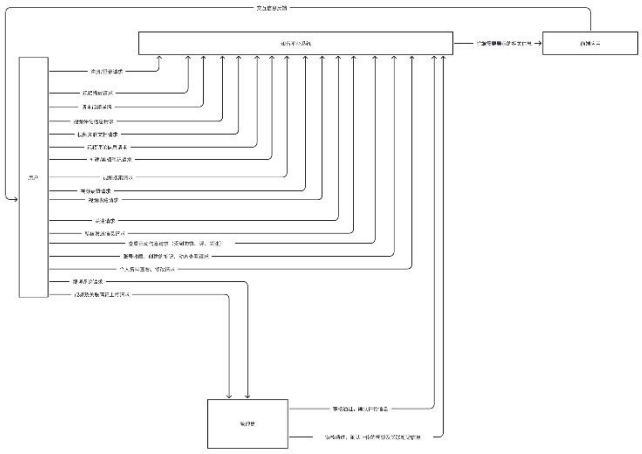


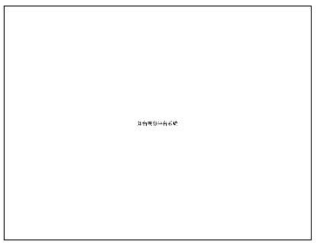
4 用例图

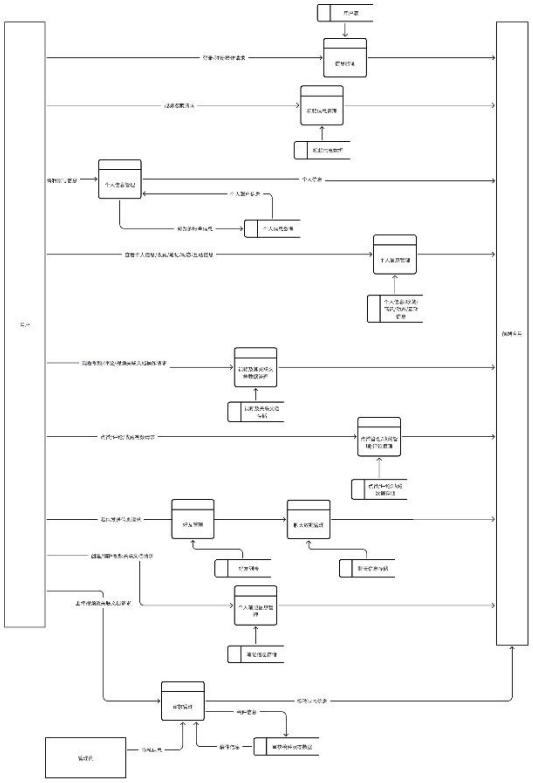

登录App
观看视频
编辑视频笔记
上传视频及关联笔记
评论
点赞
收藏
关注
聊天
管理账号信息
提交举报
审核上传的视频和笔记
处理举报
用户
管理员

5 数据流图

5.1 数据流图顶层图和底层图







6 接口设计

6.1 接口总览

6.1.1 接口命名规范

1. **URL 命名规则**：采用 RESTful 风格，使用小写字母，单词间用下划线（_）分隔，禁止使用大写字母和特殊字符（除/、?、&外）。格式为：/api/v{版本号}/{模块名}/{资源名}/{操作}（操作可选，默认通过 HTTP 方法区分）。
    1. 示例：用户注册接口/api/v1/user/register、视频上传接口/api/v1/video/upload、笔记更新接口/api/v1/note/update。
2. **接口功能对齐**：URL 需直观体现接口功能，避免模糊表述。查询类接口用 list（列表）、get（单条）；操作类接口用 create、update、delete、upload 等明确动词。
    1. 错误示例：/api/v1/user/do（功能模糊）；正确示例：/api/v1/user/get_detail（获取用户详情）。
3. **版本号强制要求**：所有接口 URL 必须包含版本号（如 v1），便于后续接口迭代和兼容性维护，禁止无版本号接口。
4. **资源复数规则**：当接口操作对象为多个资源时，资源名用复数形式；单条资源用单数形式。
    1. 示例：获取视频列表/api/v1/videos/list、获取单条视频/api/v1/video/get。

6.1.2 响应状态码规定

遵循 HTTP 标准状态码，结合业务场景扩展细分状态码，统一响应格式中的 code 字段规则：

状态码范围	含义分类	核心状态码	说明	业务场景示例
2xx	成功响应	200	操作成功	登录成功、视频播放成功
		201	资源创建成功	视频上传成功、笔记创建成功

4xx	客户端错误	400	请求参数错误	注册时手机号格式不正确
		401	未授权（未登录）	未登录时访问个人中心
		403	权限禁止	普通用户删除他人视频
		404	资源不存在	访问已删除的笔记、不存在的用户
		409	资源冲突	注册时手机号已被使用
5xx	服务器错误	500	服务器内部错误	数据库连接失败、视频处理异常
		503	服务暂不可用	推荐系统维护中、存储服务过载

- 扩展说明：响应体中 code 字段优先使用上表状态码，特殊业务场景可在对应范围下扩展细分码（如 400-1：参数缺失、400-2：参数格式错误），需在接口文档中明确说明。

6.1.3 注意事项

1. **安全性要求**：
    1. 所有接口（除公开注册、登录外）必须包含身份认证信息（Token），放在 HTTP 请求头 Authorization 字段，格式为 Bearer {Token}。
    2. 敏感数据（如密码、手机号）传输需加密，密码采用 HTTPS+哈希加盐存储，禁止明文传输。
    3. 接口需做防刷限制：同一 IP/用户 1 分钟内请求同一接口不超过 60 次，超出返回 403 状态码。
2. **兼容性要求**：
    1. 接口新增字段时需保持向后兼容，旧版本客户端可正常解析返回数据，禁止删除已有字段。

2. 日期时间格式统一为 yyyy-MM-dd HH:mm:ss（如 2025-10-15 14:30:00），时区默认 UTC+8。

3. **性能要求**：
    1. 非实时性接口（如数据统计）需支持异步处理，返回任务 ID 供客户端查询结果。
    2. 列表查询接口必须支持分页（page 页码、page_size 每页条数），默认 page=1、page_size=20，最大 page_size 不超过 100。
4. **错误处理要求**：
    1. 接口出错时，message 字段需返回用户可理解的提示（如“手机号已被注册”），同时日志中记录详细错误信息（如堆栈信息）供排查。
    2. 禁止返回敏感错误信息（如数据库表结构、服务器路径）给客户端。

6.1.4 协议约定

1. **通信协议**：全量接口采用 HTTPS 协议，确保数据传输安全，禁止使用 HTTP 协议。
2. **请求方法**：严格对应接口功能，遵循 RESTful 规范：
    1. GET：查询资源（如获取视频列表、用户详情），禁止用于数据修改操作。
    2. POST：创建资源（如注册、上传视频、发布笔记），请求参数放在请求体。
    3. PUT：全量更新资源（如更新用户资料、视频信息），需传递完整资源字段。
    4. PATCH：部分更新资源（如修改视频标题、笔记内容），仅传递需修改字段。
    5. DELETE：删除资源（如删除视频、取消关注）。
3. **数据格式**：
    1. 请求/响应数据统一使用 JSON 格式，编码为 UTF-8，禁止使用 XML、表单格式（文件上传接口除外）。
    2. 文件上传接口支持 multipart/form-data 格式，用于视频、图片等二进制文件传输。
4. **请求头规范**：
    1. 必选请求头：Content-Type: application/json（非文件上传接口）、

Authorization: Bearer {Token}（需认证接口）、User-Agent: 知音App/{版本号}。

2. 可选请求头：Accept-Language（语言偏好，默认 zh-CN）、Device-Id（设备唯一标识）。

6.2 后端接口文档

6.2.1 用户模块

6.2.1.1 用户注册

- 接口 URL：/api/v1/user/register
- 请求方法：POST
- 认证要求：无需认证
- 请求头：Content-Type: application/json
- 请求体：
- {
- "phone": "13800138000"，// 手机号（必填，11 位）
- "code": "666666"，　　// 短信验证码（必填，6 位）
- "password": "Abc123456", // 密码（必填，8-20 位，含大小写字母+数字）
- "nickname": "知识达人"，// 昵称（必填，2-10 位）
- "avatar": ""　　// 头像 URL（可选）

}

- 响应体：
- {
- "code": 201,
- "message": "注册成功",
- "data": {
- "user_id": "u123456"，// 用户唯一 ID
- "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", // 认证 Token
- "nickname": "知识达人",

- "avatar": ""
- },
- "timestamp": 1640995200

}

- 错误示例：
- {
- "code": 400-1,
- "message": "手机号格式不正确",
- "data": {},
- "timestamp": 1640995201

}

6.2.1.2 用户登录（手机号+密码）

- 接口 URL：/api/v1/user/login
- 请求方法：POST
- 认证要求：无需认证
- 请求体：
- {
- "phone": "13800138000", // 手机号（必填）
- "password": "Abc123456" // 密码（必填）

}

- 响应体：同“用户注册”成功响应
- 错误示例：
- {
- "code": 401,
- "message": "手机号或密码错误",
- "data": {},
- "timestamp": 1640995202

}

6.2.1.3 第三方登录（微信/QQ/微博）

- 接口 URL：/api/v1/user/login/third_party
- 请求方法：POST
- 认证要求：无需认证
- 请求体：
- {
- "platform": "wechat", // 平台（必填，wechat/qq/weibo）
- "code": "011aBcDeFgHiJkLmNoPqRsT23456789", // 第三方授权码（必填）
- "nickname": "微信用户", // 昵称（可选，第三方返回则可不填）
- "avatar": "" // 头像（可选，第三方返回则可不填）

}

- 响应体：同“用户注册”成功响应

6.2.1.4 获取用户详情

- 接口 URL：/api/v1/user/get_detail
- 请求方法：GET
- 认证要求：需要认证（Token）
- 请求参数（Query）：
    - user_id：目标用户 ID（可选，默认获取当前登录用户详情）
- 响应体：
- {
- "code": 200,
- "message": "success",
- "data": {
- "user_id": "u123456",
- "nickname": "知识达人",
- "avatar": "https://xxx.com/avatar.jpg",

- "bio": "分享知识，结交知音", // 个人简介
- "follow_count": 120, // 关注数
- "fan_count": 85, // 粉丝数
- "works_count": 23, // 作品数
- "tags": ["编程", "设计"] // 兴趣标签
- },
- "timestamp": 1640995203

}

6.2.1.5 更新用户资料

- 接口 URL：/api/v1/user/update_profile
- 请求方法：PATCH
- 认证要求：需要认证
- 请求体：
- {
- "nickname": "资深知识达人", // 可选
- "avatar": "https://xxx.com/new_avatar.jpg", // 可选
- "bio": "专注编程知识分享", // 可选（0-100 字）
- "tags": ["编程", "Python", "前端"] // 可选（最多 5 个）

}

- 响应体：
- {
- "code": 200,
- "message": "更新成功",
- "data": {
- "nickname": "资深知识达人",
- "avatar": "https://xxx.com/new_avatar.jpg",
- "bio": "专注编程知识分享",

- "tags": ["编程", "Python", "前端"]
- },
- "timestamp": 1640995204

}

6.2.1.6 获取短信验证码

- 接口 URL：/api/v1/user/get_sms_code
- 请求方法：POST
- 认证要求：无需认证
- 请求体：
- {
- "phone": "13800138000"，// 手机号（必填）
- "scene": "register"　　// 场景（必填，register/login/reset_pwd）

}

- 响应体：
- {
- "code": 200,
- "message": "验证码已发送至手机号 13800138000",
- "data": {},
- "timestamp": 1640995205

}

6.2.2 视频模块

6.2.2.1 视频上传（初始化）

- 接口 URL：/api/v1/video/upload/init
- 请求方法：POST
- 认证要求：需要认证
- 请求体：
- {

- "title": "Python 基础教程", // 视频标题（必填，2-50 字）
- "description": "从零开始学习 Python 语法", // 描述（可选，0-200 字）
- "tags": ["Python", "编程", "入门"], // 标签（必填，1-5 个）
- "cover_type": "auto", // 封面类型（必填，auto/custom，auto 自动生成）
- "file_size": 10485760, // 文件大小（必填，单位字节）
- "file_format": "mp4" // 文件格式（必填，mp4/mov/avi）

}

- 响应体：
- {
- "code": 200,
- "message": "初始化成功",
- "data": {
- "video_id": "v789012", // 视频唯一 ID
- "upload_url": "https://xxx.com/upload/video/v789012", // 上传地址
- "expire_time": "2025-10-15 15:30:00" // 上传地址有效期
- },
- "timestamp": 1640995206

}

6.2.2.2 视频上传（确认完成）

- 接口 URL：/api/v1/video/upload/confirm
- 请求方法：POST
- 认证要求：需要认证
- 请求体：
- {
- "video_id": "v789012", // 视频 ID（必填）
- "etag": "5F9A8B7C6D5E4F3A2B1C0D9E8F7A6B5C" // 上传文件 ETag（必填，用于校验）

}

- 响应体：
- {
- "code": 201,
- "message": "视频上传成功，正在处理",
- "data": {
- "video_id": "v789012",
- "status": "processing" // 状态：processing/completed/failed
- },
- "timestamp": 1640995207

}

6.2.2.3 获取视频详情

- 接口 URL：/api/v1/video/get_detail
- 请求方法：GET
- 认证要求：需要认证
- 请求参数（Query）：
  - video_id: "v789012"（必填，视频 ID）
- 响应体：
- {
- "code": 200,
- "message": "success",
- "data": {
- "video_id": "v789012",
- "title": "Python 基础教程",
- "description": "从零开始学习 Python 语法",
- "cover_url": "https://xxx.com/cover/v789012.jpg", // 封面 URL
- "play_url": "https://xxx.com/play/video/v789012", // 播放 URL

- "duration": 600, // 时长（秒）
- "tags": ["Python", "编程", "入门"],
- "upload_time": "2025-10-15 14:30:00",
- "author_id": "u123456",
- "author_name": "知识达人",
- "author_avatar": "https://xxx.com/avatar.jpg",
- "play_count": 1200, // 播放量
- "like_count": 85, // 点赞数
- "comment_count": 32, // 评论数
- "share_count": 15, // 分享数
- "related_document_id": "d345678" // 关联文档 ID（可选）
- },
- "timestamp": 1640995208

}

6.2.2.4 视频列表查询

- 接口 URL：/api/v1/videos/list
- 请求方法：GET
- 认证要求：需要认证
- 请求参数（Query）：
  - page: 1（必填，页码）
  - page_size: 20（必填，每页条数，1-100）
  - tag: "Python"（可选，按标签筛选）
  - sort: "latest"（可选，排序方式：latest/hot/play_count）
  - author_id: "u123456"（可选，按作者筛选）
- 响应体：
- {
- "code": 200,

- "message": "success",
- "data": {

"total": 120

7 权限设计

《知音》App 采用多维度、分层的权限控制体系，确保系统安全性和数据隐私。权限设计涵盖 API 访问控制、数据库权限管理、文件系统权限、缓存权限、消息队列权限等多个方面，并结合角色访问控制（RBAC）模型和 JWT 实现用户身份认证。

7.1 基于角色的 API 访问控制

系统采用基于角色的访问控制（RBAC）模型，结合 JWT 实现用户身份认证和 API 权限控制。用户被划分为不同的角色，每个角色拥有特定的权限集合。当用户发起请求时，系统通过 JWT Token 验证用户身份，并根据路由权限映射表检查用户是否具备相应操作权限。

系统预设了六种用户角色，形成完整的权限层级体系：

- **受限用户（Restricted User）**：违反社区规定被限制的用户，仅能浏览公开内容，无法进行评论、点赞、关注等互动操作，不能上传内容
- **普通用户（User）**：注册后的默认角色，可进行基本操作，如查看公开内容、发表评论、点赞、关注等，也可上传视频、创建文档、发布内容
- **内容创作者（Creator）**：在普通用户权限基础上，可享受专业推流服务、直播带货等特权功能，是平台内容的主要生产者
- **运营人员（Operator）**：负责平台日常运营管理，可推荐优质内容、管理话题标签、处理用户举报、执行用户限制操作、进行内容审核、管理广告投放、分析运营数据等，是平台内容生态的重要维护者
- **管理员（Admin）**：具备较高的管理权限，可管理用户账户、审核内容、配置系统参数、查看运营数据、处理系统故障、管理权限分配、监控系统性能等，是系统稳定运行的重要保障者
- **超级管理员（Super Admin）**：拥有系统最高权限，可进行所有操作，包括角色权限配置、系统核心设置、数据库管理、系统架构调整、安全策略制定、审计日志管理等，是系统安全和稳定的核心保障者

RBAC 实现方式：

系统采用自定义的 RBAC 实现机制，通过权限策略表来管理角色与资源的访问关系。权限策略以"(角色, 资源, 操作)"三元组的形式存储在数据库中，其中资源可以是 API 端点、数据表或功能模块，操作包括读、写、删除等。当用户发起请求时，系统首先验证 JWT Token 获取用户角色，然后查询权限策略表确认该角色是否具备访问请求资源的权限。

为提高权限验证效率，系统采用以下优化策略：

1. **缓存机制**：将用户角色信息和权限策略缓存到 Redis 中，减少数据库查询次数

2. **策略预加载**：在服务启动时预加载热点权限策略到内存中

3. **批量验证**：对批量请求进行权限验证时，合并相同角色的权限检查

RBAC 实现方案选择：

根据不同阶段的业务需求和流量规模，系统支持灵活的 RBAC 实现方案切换：

- **前期小流量阶段**：可采用通用权限管理框架，其提供了丰富的权限模型和易于使用的 API，便于快速开发和验证业务逻辑

- **后期大流量阶段**：采用基于 Redis 的自定义权限控制系统，通过精细的缓存策略和定制化实现，满足高性能、低延迟的权限验证需求

API 权限验证流程如下：

用户发起请求首先到达 API 网关，网关检查请求路径是否需要权限验证。如需验证，则从请求头中提取 JWT Token 并解析获取用户 ID 和角色信息，然后根据路由权限映射表验证用户角色是否满足要求。权限验证通过后转发请求到对应的服务，验证失败则返回 403 Forbidden 错误。

主要 API 路由权限包括：

- 用户注册和登录接口对所有人开放
- 个人资料查看和修改仅限用户本人
- 内容上传、编辑、删除需普通用户及以上权限
- 评论发表和删除需普通用户及以上权限
- 推流服务、直播带货等特权功能需内容创作者及以上权限

- 运营管理接口（内容推荐、标签管理、用户限制、内容审核、广告管理、数据分析等）需运营人员及以上权限
- 管理员接口（用户管理、内容审核、系统配置、权限分配、系统监控等）需管理员及以上权限
- 系统配置、权限管理、安全策略、审计日志等核心接口仅限超级管理员
- 受限用户仅能访问公开内容浏览接口，无法访问互动和内容创作相关接口

7.2 数据库权限管理

数据库权限采用多层级控制策略，通过数据库用户角色、表级权限和行级权限三个维度实现精细化权限管理。

数据库用户角色定义：

- **app_restricted**：受限用户角色，仅具备对用户、内容相关表的只读权限，无法访问社交相关表

- **app_user**：普通用户角色，具备对用户、内容、社交相关表的只读权限，并可对自己创建的数据进行增删改操作

- **app_creator**：内容创作者角色，在 app_user 基础上可享受专业内容服务的特殊权限

- **app_operator**：运营人员角色，可对内容表进行推荐标记操作，对用户表进行状态更新操作，可访问运营数据表进行数据分析

- **app_admin**：管理员角色，可对所有表进行增删改查操作

- **app_super**：超级管理员角色，拥有数据库的所有权限，包括结构修改权限、安全策略配置权限、审计日志管理权限

数据表权限控制策略：

用户表权限根据数据敏感性进行分级控制，受限用户仅能访问公开的用户信息，普通用户可访问自己的信息和公开信息，并可创建内容相关数据，运营人员可更新用户状态（如限制、解限等），管理员可访问所有用户信息并可进行权限管理，超级管理员可修改所有用户信息、调整系统架构、制定安全策略。内容表权限遵循创建者拥有最高权限的原则，普通用户可操作自己创建的数据，内容创作者除拥有普通用户的所有权限外，还可享受专业内容服务的特殊权限，运营人员可进行内容推荐标记，管理员可审核和管理所有内容、监控内容质量，超级管理员可删除任何内容、管理审计日志。社交表权限同样遵循用户自主原则，受限用户无法进行社交操作，普通用户可操作自

己的社交数据，管理员可处理违规内容并可监控社区健康度，超级管理员可进行全局社交数据管理。

为实现更精细的权限控制，系统还采用了数据行级权限控制机制：

- 通过数据库视图实现行级权限控制，确保用户只能访问授权的数据行
- 使用存储过程控制数据访问权限，提高权限验证的效率和安全性
- 在应用层实现数据过滤机制，作为数据库权限控制的补充

7.3 文件系统权限控制

文件系统权限根据文件类型和用户角色进行分类管理，确保文件访问的安全性。

上传文件权限：

受限用户无法上传任何文件，普通用户可上传个人头像、视频文件和文档文件，内容创作者除拥有普通用户的所有上传权限外，还可享受专业推流服务、直播带货等特权功能，运营人员可上传运营相关文件，管理员可管理所有上传文件。

文件访问权限：

公开文件对所有人开放访问，私有文件仅创建者和管理员可访问，付费文件仅购买用户可访问，运营文件仅运营人员及以上角色可访问。

7.4 缓存和消息队列权限

缓存权限：

Redis 中的用户会话数据仅用户本人可访问，公共缓存数据对所有用户开放，运营缓存数据仅运营人员及以上角色可访问，管理员缓存数据仅管理员可访问，系统监控缓存数据仅管理员及以上角色可访问。

消息队列权限：

Kafka 主题权限根据业务需求进行分配，用户行为主题允许所有服务写入但仅推荐服务可消费，通知主题仅消息服务可写入且仅通知服务可消费，运营管理主题仅限运营人员及以上角色访问，管理主题仅限管理员服务访问。

7.5 安全增强机制

敏感操作二次验证：

对于修改密码、删除内容、权限变更、支付操作等敏感操作，系统要求进行二次验证，如原密码验证、二次确认、管理员审批或短信验证码验证等。

8 安全性、健壮性与可扩展性

8.1 安全性

（一）安全需求定义

基于需求文档，“知音”App 的安全需求需覆盖三大核心场景：一是**用户数据安全**，包括密码、手机号等敏感信息的传输与存储保护，以及用户隐私数据的授权访问控制；二是**内容安全**，所有用户上传的视频、文档需符合国家法律法规，需建立审核与举报机制；三是**系统安全**，所有 API 接口需身份验证，防范 DDoS、SQL 注入等常见网络攻击，同时保障核心功能（如推荐系统、学习记录）的访问权限合规。例如，内容创作者上传的技术文档需经过安全检测，避免包含恶意代码；内容消费者的笔记数据仅本人及授权用户可访问。

（二）风险评估与应对

1. **数据泄露风险**：用户密码、学习记录等核心数据若未加密，可能导致信息泄露。应对措施：采用 SHA-256 加盐哈希存储密码，手机号、等敏感信息传输时使用 TLS 1.3 加密，符合需求文档中“所有敏感数据（如密码、手机号）在传输和存储时必须加密”的要求；同时，对用户表、学习记录表等核心数据表设置字段级权限，限制非授权访问。
2. **账号盗用风险（可选）**：仅依赖手机号+验证码登录，可能面临短信劫持导致的账号被盗问题。应对措施：增加异常登录检测功能，当系统检测到陌生设备、异地 IP 登录时，触发二次验证（如安全问题+验证码），并实时推送登录提醒至用户预留邮箱，符合“用户安全需包含异常登录检测”的需求。
3. **内容违规风险（可选）**：用户上传的视频、文档可能包含违规信息，违反合规约束。应对措施：搭建“AI 初审+人工复审”机制，AI 自动识别违规内容（如敏感词、违规画面），人工对疑似违规内容进行复核；同时开放用户举报入口，支持对违规内容标记，审核通过后立即下架，满足“所有内容需符合国家相关法律法规，需建立内容审核机制”的要求。

4. **API 攻击风险**：未授权的 API 调用或恶意请求（如 DDoS、SQL 注入）可能导致系统瘫痪。应对措施：所有 RESTful API 接口需携带 JWT 令牌进行身份验证，令牌有效期设为 2 小时；对接口请求频率限流（单用户单接口每秒最多 5 次请求），采用预编译 SQL 语句防范 SQL 注入，配置 WAF（Web 应用防火墙）拦截异常请求，落实“所有 API 接口必须进行身份验证和权限控制，系统需具备防范常见网络攻击的能力”的需求。

8.2 健壮性

（一）健壮性需求明确

结合“知音”App 的业务场景与非功能需求，健壮性需求需聚焦以下核心目标：一是**高可用性**，系统核心功能（用户登录、视频播放、笔记创作）可用性强，故障概率小；二是**容错能力**，在服务异常（如推荐服务宕机）、网络波动（如弱网环境）、数据异常（如视频文件损坏）场景下，系统需优雅降级，避免崩溃；三是**性能稳定性**，满足“95%的 API 请求在 500ms 内响应”“视频首帧加载时间（Wi-Fi 下）小于 1s”“App 冷启动时间小于 3s”的性能指标，且在并发量达到 1000 QPS 时仍保持稳定。

（二）容错设计

1. **服务容错机制**：采用 Sentinel 实现微服务的限流、熔断与降级。当推荐服务因高并发出现响应延迟时，触发限流策略，限制每秒请求量不超过服务最大处理能力；当服务出现异常（如返回错误码 500）次数超过阈值时，自动熔断（熔断时长 5 分钟），期间请求导向降级接口（如返回热门视频列表而非个性化推荐）；视频存储服务异常时，降级为仅展示已缓存的视频缩略图，避免整个 Feed 流功能不可用，保障核心体验。

2. **数据容错设计**：数据库采用 MySQL 主从复制架构，主库负责写操作，从库负责读操作，当主库故障时，通过 MGR（MySQL Group Replication）自动切换至从库，切换时间不超过 30 秒；视频、文档文件存储于 MinIO 分布式对象存储，默认配置 3 副本，当单个存储节点故障时，其他副本可正常提供服务，避免文件丢失；用户提交的笔记内容实时保存至本地缓存，若网络中断，待网络恢复后自动同步至服务器，防止数据丢失。

3. **异常处理规范**：统一前后端异常处理流程，服务端定义标准化错误码（参考需求文档中“40001 表示参数错误，50001 表示数据库错误”的规范），返回包含“code+message+data+timestamp”的 JSON 格式响应；客户端接收异常后，根据错误码展示友好提示（如参数错误提示“请输入正确的视频标题”，数据库错误提示“系统暂时异常，请稍后重试”），同时记录异常日志至本地，便于问题排查，避免 App 闪退。

（三）资源管理

1. **内存管理**：客户端（Flutter）采用图片懒加载、视频帧预加载策略，仅加载当前屏幕可见的视频缩略图与笔记内容，滚动时回收不可见资源，避免内存泄漏；服务端（Go 微服务）通过协程池控制并发数（默认每服务实例协程数不超过 1000），防止协程泄露导致内存占用过高；对大文件（如超过 200MB 的视频）采用分片上传/下载，避免单次请求占用过多内存。

2. **CPU 资源管理**：服务端针对 CPU 密集型任务（如视频转码、全文检索），采用异步处理模式，通过 Kafka 消息队列将任务分发至专用服务节点，避免占用核心业务（如 API 请求处理）的 CPU 资源；客户端优化 UI 渲染逻辑，减少重绘与重排，例如视频播放页采用硬件加速渲染，降低 CPU 使用率，保障播放流畅性。

3. **网络资源管理**：针对弱网环境，客户端实现视频断点续传（基于 HTTP Range 请求），支持缓存已浏览的视频（默认缓存 7 天）与文档内容，弱网时优先加载缓存内容；服务端优化 API 返回数据结构，移除冗余字段（如列表查询仅返回必要的视频 ID、标题、封面 URL，详情查询再返回完整信息），降低网络传输压力；采用 CDN（内容分发网络）加速视频、图片等静态资源分发，将资源节点部署至全国主要城市，减少用户访问延迟。

4. **数据库资源管理**：对写密集型表（如 likes 表、learning_records 表），仅建立必要的索引（如 likes 表建立 user_id+video_id 复合索引），避免过多索引影响写入性能；对读密集型表（如 videos 表、users 表），建立复合索引（如 videos 表建立 tag_id+created_at 复合索引）提升查询效率；采用 Redis 缓存热点数据（如用户信息、热门视频列表），缓存有效期设为 1 小时，降低数据库访问压力；设置数据库连接池（默认最大连接数 100），避免连接数过多导致数据库崩溃。

8.3 可扩展性

（一）识别扩展点

结合“知音”App 的业务演进规划（从 V1.0 MVP 到 V2.0 商业化），核心扩展点包括：一是**功能扩展**，V1.0 需实现用户系统、视频管理、推荐系统、笔记功能等 P0 级核心功能，未来需扩展 P1 级（基础社交、圈子功能）、P2 级（视频编辑、数据统计）功能，以及 V2.0 的付费专栏、直播、创作者激励等商业化功能；二是**性能扩展**，可能未来需支撑 10000 QPS、千万级视频数据的访问需求；三是**多端扩展**，当前聚焦移动端（iOS 13.0+、Android 8.0+），未来可能扩展 Web 端、平板端，满足不同场景的使用需求；四是**第三方集成**，需求文档中预留了微信/GitHub 第三方登录、支付宝/微信支付接口，未来需集成这些第三方服务，丰富平台能力。

（二）模块化设计

1. **微服务拆分**：按业务域将系统拆分为独立的微服务，包括用户服务（负责注册

登录、个人资料管理）、内容服务（负责视频/文档上传、存储、管理）、推荐服务（负责 Feed 流推荐、个性化内容分发）、社交服务（负责关注、评论、私信）、笔记服务（负责笔记创建、管理、同步）、审核服务（负责内容审核、违规处理）。各服务间通过 RPC 通信，接口定义遵循统一规范（如 Protobuf），服务内部高内聚、服务间低耦合，未来新增功能（如直播）时，可独立开发直播服务，通过调用用户服务的认证接口、内容服务的存储接口快速集成，无需修改现有服务代码。

2. **前端模块化**：客户端（Flutter）采用模块化架构，将核心功能拆分为基础模块（路由、网络、存储）、业务模块（用户模块、视频模块、笔记模块、社交模块）、插件模块（视频编辑插件、第三方登录插件）。模块间通过接口通信，依赖倒置，例如视频模块不直接依赖推荐模块，而是通过推荐接口获取内容，未来替换推荐逻辑时无需修改视频模块代码；非核心功能（如视频编辑）设计为插件，支持按需下载安装，V1.2 版本上线时，用户无需更新整个 App，仅需下载插件即可使用，降低更新成本。

3. **数据库模块化**：采用分库分表策略，按业务域拆分数据库（用户库、内容库、社交库、笔记库），避免单库数据量过大；针对大表（如 videos 表、comments 表），按时间（如每月分表）或哈希（如用户 ID 哈希分表）进行分表，未来数据量增长时，可通过新增分表扩展存储能力；数据库表设计预留扩展字段（如 users 表预留“member_level”字段用于未来会员体系，videos 表预留“pay_status”字段用于付费内容），避免新增字段时修改表结构。

（三）技术选型

1. **后端技术选型**：采用 Go 语言（Hertz 框架）作为后端开发语言，其高性能、高并发能力适合微服务架构，可支撑未来高 QPS 需求；数据库选用 MySQL（主从复制+分库分表）存储结构化数据，Redis 缓存热点数据，MinIO 存储视频、文档等非结构化数据，三者均支持水平扩展，可通过新增节点提升存储与处理能力；消息队列选用 Kafka，高吞吐量适合处理实时数据流（如用户行为日志、视频转码任务），支持集群部署，可通过新增 Broker 节点扩展消息处理能力；搜索引擎选用 Elasticsearch，支持全文搜索与个性化推荐，集群部署可扩展搜索性能，满足未来千万级内容的搜索需求，符合需求文档中“技术栈需采用 Go 微服务、Kubernetes 等”的约束。

2. **前端技术选型**：采用 Flutter 作为跨平台开发框架，可同时支持 iOS（13.0+）、Android（8.0+）平台，开发效率高，性能接近原生，未来扩展 Web 端、平板端时，可复用大部分业务逻辑，仅需调整 UI 布局，降低多端开发成本；状态管理采用 Bloc 模式，实现业务逻辑与 UI 分离，便于功能扩展与维护；网络请求采用 Dio 框架，支持拦截器、请求重试、断点续传，可适配未来复杂的网络场景（如多域名请求、第三方 API 调用）。

3. **部署与运维技术选型**：采用 Docker 容器化部署应用，docker compose 进行容

器编排，支持服务自动扩缩容、滚动更新、故障自愈，未来业务增长时，可通过增加 Pod 实例或节点扩展服务能力；监控系统选用 Prometheus+Grafana，支持多维度指标监控与可视化，可扩展监控维度（如未来新增直播服务的监控指标）；日志系统选用 ELK，支持全链路日志收集与分析，可扩展日志存储与查询能力，满足未来大规模系统的运维需求，符合“云原生部署（Docker、K8s）”的技术实践目标。